15

Routing: Exterior Gateway Protocols And Autonomous Systems (BGP)

15.1 Introduction

The previous chapter introduces the idea of route propagation and examines one protocol routers use to exchange routing information. This chapter extends our understanding of internet routing architectures. It discusses the concept of autonomous systems, and shows a protocol that a group of networks and routers operating under one administrative authority uses to propagate routing information about its networks to other groups.

15.2 Adding Complexity To The Architectural Model

The original core routing system evolved at a time when the Internet had a single wide area backbone as the previous chapter describes. Consequently, part of the motivation for a core architecture was to provide connections between a network at each site and the backbone. If an internet consists of only a single backbone plus a set of attached local area networks, the core approach propagates all necessary routing information correctly. Because all routers attach to the wide area backbone network, they can exchange all necessary routing information directly. Each router knows the single local network to which it attaches, and propagates that information to the other routers. Each router learns about other destination networks from other routers.

It may seem that it would be possible to extend the core architecture to an arbitrary size internet merely by adding more sites, each with a router connecting to the backbone. Unfortunately, the scheme does not scale — having all routers participate in a single routing protocol only suffices for trivial size internets. There are three reasons. First, even if each site consists of a single network, the scheme cannot accommodate an arbitrary number of sites because each additional router generates routing traffic. If a large set of routers attempt to communicate, the total bandwidth becomes overwhelming. Second, the scheme cannot accommodate multiple routers and networks at a given site because only those routers that connect directly to the backbone network can communicate directly. Third, in a large internet, the networks and routers are not all managed by a single entity, nor are shortest paths always used. Instead, because networks are owned and managed by independent groups, the groups may choose policies that differ. A routing architecture must provide a way for each group to independently control routing and access.

The consequences of limiting router interaction are significant. The idea provides the motivation for much of the routing architecture used in the global Internet, and explains some of the mechanisms we will study. To summarize this important principle:

> *Although it is desirable for routers to exchange routing information, it is impractical for all routers in an arbitrarily large internet to participate in a single routing update protocol.*

15.3 Determining A Practical Limit On Group Size

The above statement leaves many questions open. For example, what size internet is considered "large"? If only a limited set of routers can participate in an exchange of routing information, what happens to routers that are excluded? Do they function correctly? Can a router that is not participating ever forward a datagram to a router that is participating? Can a participating router forward a datagram to a non-participating router?

The answer to the question of size involves understanding the algorithm being used and the capacity of the network that connects the routers as well as the details of the routing protocol. There are two issues: delay and overhead. Delay is easy to understand. For example, consider the maximum delay until all routers are informed about a change when they use a distance-vector protocol. Each router must receive the new information, update its routing table, and then forward the information to its neighbors. In an internet with N routers arranged in a linear topology, N steps are required. Thus, N must be limited to guarantee rapid distribution of information.

The issue of overhead is also easy to understand. Because each router that participates in a routing protocol must send messages, a larger set of participating routers means more routing traffic. Furthermore, if routing messages contain a list of possible destinations, the size of each message grows as the number of routers and networks in-

crease. To ensure that routing traffic remains a small percentage of the total traffic on the underlying networks, the size of routing messages must be limited.

In fact, most network managers do not have sufficient information required to perform detailed analysis of the delay or overhead. Instead, they follow a simple heuristic guideline:

> *It is safe to allow up to a dozen routers to participate in a single routing information protocol across a wide area network; approximately five times as many can safely participate across a set of local area networks.*

Of course, the rule only gives general advice and there are many exceptions. For example, if the underlying networks have especially low delay and high capacity, the number of participating routers can be larger. Similarly, if the underlying networks have unusually low capacity or a high amount of traffic, the number of participating routers must be smaller to avoid overloading the networks with routing traffic.

Because an internet is not static, it can be difficult to estimate how much traffic routing protocols will generate or what percentage of the underlying bandwidth the routing traffic will consume. For example, as the number of hosts on a network grows over time, increases in the traffic generated consume more of the network capacity. In addition, increased traffic can arise from new applications. Therefore, network managers cannot rely solely on the guideline above when choosing a routing architecture. Instead, they usually implement a *traffic monitoring* scheme. In essence, a traffic monitor listens passively to a network and records statistics about the traffic. In particular, a monitor can compute both the network utilization (i.e., percentage of the underlying bandwidth being used) and the percentage of packets carrying routing protocol messages. A manager can observe traffic trends by taking measurements over long periods (e.g., weeks or months), and can use the output to determine whether too many routers are participating in a single routing protocol.

15.4 A Fundamental Idea: Extra Hops

Although the number of routers that participate in a single routing protocol must be limited, doing so has an important consequence because it means that some routers will be outside the group. It might seem that an "outsider" could merely make a member of the group a default. In the early Internet, the core system did indeed function as a central routing mechanism to which noncore routers sent datagrams for delivery. However, researchers learned an important lesson: if a router outside of a group uses a member of the group as a default route, routing will be suboptimal. More important, one does not need a large number of routers or a wide area network — the problem can occur whenever a nonparticipating router uses a participating router for delivery. To see why, consider the example in Figure 15.1.

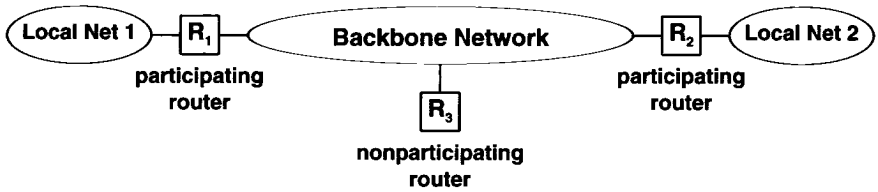


Figure 15.1 An architecture that can cause the extra hop problem. Nonoptimal routing occurs when a nonparticipating router connected to the backbone has a default route to a participating router.

In the figure, routers R_1 and R_2 connect to local area networks *1* and *2*, respectively. Because they participate in a routing protocol, they both know how to reach both networks. Suppose nonparticipating router R_3 chooses one of the participating routers, say R_1, as a default. That is, R_3 sends R_1 all datagrams destined for networks to which it has no direct connection. In particular, R_3 sends datagrams destined for network *2* across the backbone to its chosen participating router, R_1, which must then forward them back across the backbone to router R_2. The optimal route, of course, requires R_3 to transmit datagrams destined for network *2* directly to R_2. Notice that the choice of participating router makes no difference. Only destinations that lie beyond the chosen router have optimal routes; all paths that go through other backbone routers require the datagram to make a second, unnecessary trip across the backbone network. Also notice that the participating routers cannot use ICMP redirect messages to inform R_3 that it has nonoptimal routes because ICMP redirect messages can only be sent to the original source and not to intermediate routers.

We call the routing anomaly illustrated in Figure 15.1 the *extra hop problem.* The problem is insidious because everything appears to work correctly — datagrams do reach their destination. However, because routing is not optimal, the system is extremely inefficient. Each datagram that takes an extra hop consumes resources on the intermediate router as well as twice as much backbone bandwidth as it should. Solving the problem requires us to change our view of architecture:

> *Treating a group of routers that participate in a routing update protocol as a default delivery system can introduce an extra hop for datagram traffic; a mechanism is needed that allows nonparticipating routers to learn routes from participating routers so they can choose optimal routes.*

15.5 Hidden Networks

Before we examine mechanisms that allow a router outside a group to learn routes, we need to understand another aspect of routing: hidden networks (i.e. networks that are concealed from the view of routers in a group). Figure 15.2 shows an example that will illustrate the concept.

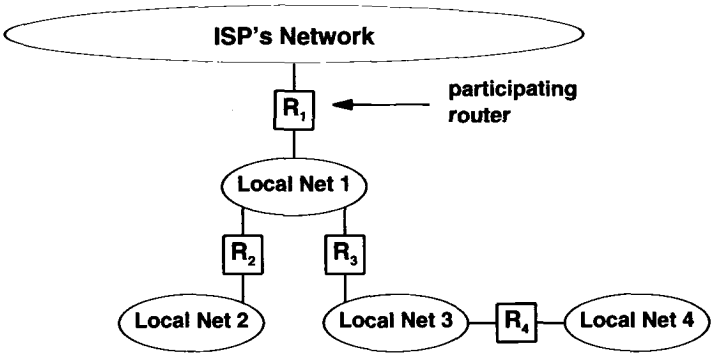


Figure 15.2 An example of multiple networks and routers with a single backbone connection. A mechanism is needed to pass reachability information about additional local networks into the core system.

In the figure, a site has multiple local area networks with multiple routers connecting them. Suppose the site has just installed local network *4* and has obtained an Internet address for it (for now, imagine that the site obtained the address from another ISP). Also assume that the routers R_2, R_3, and R_4 each have correct routes for all four of the site's local networks as well as a default route that passes other traffic to the ISP's router, R_1. Hosts directly attached to local network *4* can communicate with one another, and any computer on that network can route packets out to other Internet sites. However, because router R_1 attaches only to local network *1*, it does not know about local network *4*. We say that, from the viewpoint of the ISP's routing system, local network *4* is *hidden* behind local network *1*. The important point is:

> *Because an individual organization can have an arbitrarily complex set of networks interconnected by routers, no router from another organization can attach directly to all networks. A mechanism is needed that allows nonparticipating routers to inform the other group about hidden networks.*

We now understand a fundamental aspect of routing: information must flow in two directions. Routing information must flow from a group of participating routers to a nonparticipating router, and a nonparticipating router must pass information about hidden networks to the group. Ideally, a single mechanism should solve both problems. Building such a mechanism can be tricky. The subtle issues are responsibility and capability. Exactly where does responsibility for informing the group reside? If we decide that one of the nonparticipating routers should inform the group, which one is capable of doing it? Look again at the example. Router R_4 is the router most closely associated with local network *4*, but it lies *2* hops away from the nearest core router. Thus, R_4 must depend on router R_3 to route packets to network *4*. The point is that R_4 knows about local network *4* but cannot pass datagrams directly from R_1. Router R_3 lies one hop from the core and can guarantee to pass packets, but it does not directly attach to local network *4*. So, it seems incorrect to grant R_3 responsibility for advertising network *4*. Solving this dilemma will require us to introduce a new concept. The next sections discuss the concept and a protocol that implements it.

15.6 Autonomous System Concept

The puzzle over which router should communicate information to the group arises because we have only considered the mechanics of an internet routing architecture and not the administrative issues. Interconnections, like those in the example of Figure 15.2, that arise because an internet has a complex structure, should not be thought of as multiple independent networks connected to an internet. Instead, the architecture should be thought of as a single organization that has multiple networks under its control. Because the networks and routers fall under a single administrative authority, that authority can guarantee that internal routes remain consistent and viable. Furthermore, the administrative authority can choose one of its routers to serve as the machine that will apprise the outside world of networks within the organization. In the example from Figure 15.2, because routers R_2, R_3, and R_4 fall under control of one administrative authority, that authority can arrange to have R_3 advertise networks *2*, *3*, and *4* (R_1 already knows about network *1* because it has a direct connection to it).

For purposes of routing, a group of networks and routers controlled by a single administrative authority is called an *autonomous system* (*AS*). Routers within an autonomous system are free to choose their own mechanisms for discovering, propagating, validating, and checking the consistency of routes. Note that, under this definition, the original Internet core routers formed an autonomous system. Each change in routing protocols within the core autonomous system was made without affecting the routers in other autonomous systems. In the previous chapter, we said that the original Internet core system used GGP to communicate, and a later generation used SPREAD. Eventually, ISPs evolved their own backbone networks that use more recent protocols. The next chapter reviews some of the protocols that autonomous systems use internally to propagate routing information.

15.7 From A Core To Independent Autonomous Systems

Conceptually, the autonomous system idea was a straightforward and natural generalization of the original Internet architecture, depicted by Figure 15.2, with autonomous systems replacing local area networks. Figure 15.3 illustrates the idea.

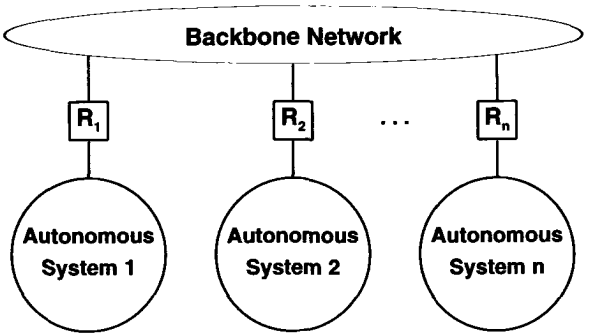


Figure 15.3 Architecture of an internet with autonomous systems at backbone sites. Each autonomous system consists of multiple networks and routers under a single administrative authority.

To make networks that are hidden inside autonomous systems reachable throughout the Internet, each autonomous system must advertise its networks to other autonomous systems. An advertisement can be sent to any autonomous system. In a centralized, core architecture, however, it is crucial that each autonomous system propagate information to one of the routers in the core autonomous system.

It may seem that our definition of an autonomous system is vague, but in practice the boundaries between autonomous systems must be precise to allow automated algorithms to make routing decisions. For example, an autonomous system owned by a corporation may choose not to route packets through an autonomous system owned by another even though they connect directly. To make it possible for automated routing algorithms to distinguish among autonomous systems, each is assigned an *autonomous system number* by the central authority that is charged with assigning all Internet network addresses. When routers in two autonomous systems exchange routing information, the protocol arranges for messages to carry the autonomous system number of the system each router represents.

We can summarize the ideas:

> *A large TCP/IP internet has additional structure to accommodate administrative boundaries: each collection of networks and routers managed by one administrative authority is considered to be a single autonomous system that is free to choose an internal routing architecture and protocols.*

We said that an autonomous system needs to collect information about all its networks and designate one or more routers to pass the information to other autonomous systems. The next sections presents the details of a protocol routers use to advertise network reachability. Later sections return to architectural questions to discuss an important restriction the autonomous system architecture imposes on routing.

15.8 An Exterior Gateway Protocol

Computer scientists use the term *Exterior Gateway Protocol* (*EGP*)† to denote any protocol used to pass routing information between two autonomous systems. Currently a single exterior protocol is used in most TCP/IP internets. Known as the *Border Gateway Protocol* (*BGP*), it has evolved through four (quite different) versions. Each version is numbered, which gives rise to the formal name of the current version: *BGP-4*. Throughout this text, the term *BGP* will refer to *BGP-4*.

When a pair of autonomous systems agree to exchange routing information, each must designate a router‡ that will speak BGP on its behalf; the two routers are said to become *BGP peers* of one another. Because a router speaking BGP must communicate with a peer in another autonomous system, it makes sense to select a machine that is near the "edge" of the autonomous system. Hence, BGP terminology calls the machine a *border gateway* or *border router*. Figure 15.4 illustrates the idea.

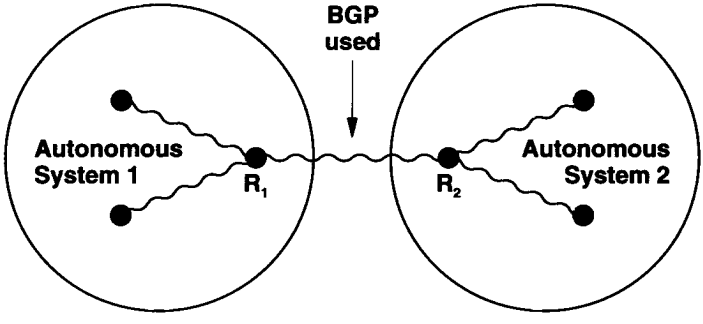


Figure 15.4 Conceptual illustration of two routers, R_1 and R_2, using BGP to advertise networks in their autonomous systems after collecting the information from other routers internally. An organization using BGP usually chooses a router that is close to the outer "edge" of the autonomous system.

In the figure, router R_1 gathers information about networks in autonomous system *1* and reports that information to router R_2 using BGP, while router R_2 reports information from autonomous system *2*.

†Originally, the term *EGP* referred to a specific protocol that was used for communication with the Internet core; the name was coined when the term *gateway* was used instead of *router*.

‡Although the protocol allows an arbitrary computer to be used, most autonomous systems run BGP on a router; all the examples in this text will assume BGP is running on a router.

15.9 BGP Characteristics

BGP is unusual in several ways. Most important, BGP is neither a pure distance-vector protocol nor a pure link state protocol. It can be characterized by the following:

Inter-Autonomous System Communication. Because BGP is designed as an exterior gateway protocol, its primary role is to allow one autonomous system to communicate with another.

Coordination Among Multiple BGP Speakers. If an autonomous system has multiple routers each communicating with a peer in an outside autonomous system, BGP can be used to coordinate among routers in the set to guarantee that they all propagate consistent information.

Propagation Of Reachability Information. BGP allows an autonomous system to advertise destinations that are reachable either in or through it, and to learn such information from another autonomous system.

Next-Hop Paradigm. Like distance-vector routing protocols, BGP supplies *next hop* information for each destination.

Policy Support. Unlike most distance-vector protocols that advertise exactly the routes in the local routing table, BGP can implement policies that the local administrator chooses. In particular, a router running BGP can be configured to distinguish between the set of destinations reachable by computers inside its autonomous system and the set of destinations advertised to other autonomous systems.

Reliable Transport. BGP is unusual among protocols that pass routing information because it assumes reliable transport. Thus, BGP uses TCP for all communication.

Path Information. In addition to specifying destinations that can be reached and a next hop for each, BGP advertisements include path information that allows a receiver to learn a series of autonomous systems along a path to the destination.

Incremental Updates. To conserve network bandwidth, BGP does not pass full information in each update message. Instead, full information is exchanged once, and then successive messages carry incremental changes called *deltas*.

Support For Classless Addressing. BGP supports CIDR addresses. That is, rather than expecting addresses to be self-identifying, the protocol provides a way to send a mask along with each address.

Route Aggregation. BGP conserves network bandwidth by allowing a sender to aggregate route information and send a single entry to represent multiple, related destinations.

Authentication. BGP allows a receiver to authenticate messages (i.e., verify the identity of a sender).

15.10 BGP Functionality And Message Types

BGP peers perform three basic functions. The first function consists of initial peer acquisition and authentication. The two peers establish a TCP connection and perform a message exchange that guarantees both sides have agreed to communicate. The second function forms the primary focus of the protocol — each side sends positive or negative reachability information. That is, a sender can advertise that one or more destinations are reachable by giving a next hop for each, or the sender can declare that one or more previously advertised destinations are no longer reachable. The third function provides ongoing verification that the peers and the network connections between them are functioning correctly.

To handle the three functions described above, BGP defines four basic message types. Figure 15.5 contains a summary.

Type Code	Message Type	Description
1	OPEN	Initialize communication
2	UPDATE	Advertise or withdraw routes
3	NOTIFICATION	Response to an incorrect message
4	KEEPALIVE	Actively test peer connectivity

Figure 15.5 The four basic message types in BGP-4.

15.11 BGP Message Header

Each BGP message begins with a fixed header that identifies the message type. Figure 15.6 illustrates the header format.

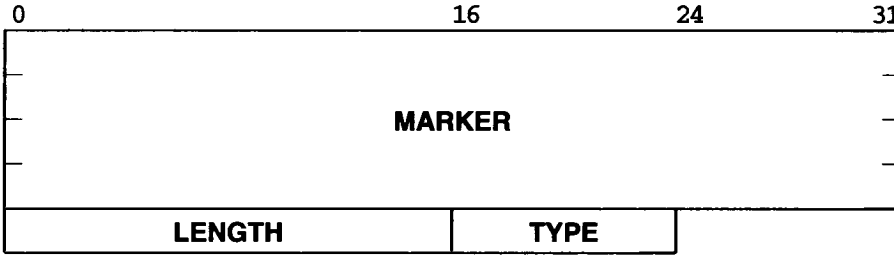


Figure 15.6 The format of the header that precedes every BGP message.

The 16-octet *MARKER* field contains a value that both sides agree to use to mark the beginning of a message. The 2-octet *LENGTH* field specifies the total message length measured in octets. The minimum message size is *19* octets (for a message type that has no data following the header), and the maximum allowable length is *4096* oc-

tets. Finally, the 1-octet *TYPE* field contains one of the four values for the message type listed in Figure 15.5.

The *MARKER* may seem unusual. In the initial message, the marker consists of all 1s; if the peers agree to use an authentication mechanism, the marker can contain authentication information. In any case, both sides must agree on the value so it can be used for *synchronization*. To understand why synchronization is necessary, recall that all BGP messages are exchanged across a stream transport (i.e., TCP), which does not identify the boundary between one message and the next. In such an environment, a simple error on either side can have dramatic consequences. In particular, if either the sender or receiver miscounts the octets in a message, a *synchronization error* will occur. More important, because the transport protocol does not specify message boundaries, the transport protocol will not alert the receiver to the error. Thus, to ensure that the sender and receiver remain synchronized, BGP places a well-known sequence at the beginning of each message, and requires a receiver to verify that the value is intact before processing the message.

15.12 BGP OPEN Message

As soon as two BGP peers establish a TCP connection, they each send an *OPEN* message to declare their autonomous system number and establish other operating parameters. In addition to the standard header, an *OPEN* message contains a value for a *hold timer* that is used to specify the maximum number of seconds which may elapse between the receipt of two successive messages. Figure 15.7 illustrates the format.

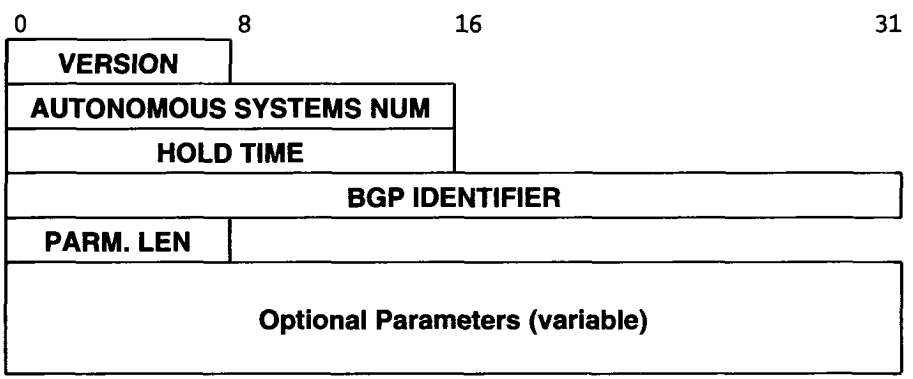


Figure 15.7 The format of the BGP OPEN message that is sent at startup. These octets follow the standard message header.

Most fields are straightforward. The *VERSION* field identifies the protocol version used (this format is for version 4). Recall that each autonomous system is assigned a unique number. Field *AUTONOMOUS SYSTEMS NUM* gives the autonomous system

number of the sender's system. The *HOLD TIME* field specifies a maximum time that the receiver should wait for a message from the sender. The receiver is required to implement a timer using this value. The timer is reset each time a message arrives; if the timer expires, the receiver assumes the sender is no longer available (and stops forwarding datagrams along routes learned from the sender).

Field *BGP IDENTIFIER* contains a 32-bit integer value that uniquely identifies the sender. If a machine has multiple peers (e.g., perhaps in multiple autonomous systems), the machine must use the same identifier in all communication. The protocol specifies that the identifier is an IP address. Thus, a router must choose one of its IP addresses to use with all BGP peers.

The last field of an *OPEN* message is optional. If present, field *PARM. LEN* specifies the length measured in octets, and the field labeled *Optional Parameters* contains a list of parameters. It has been labeled *variable* to indicate that the size varies from message to message. When parameters are present, each parameter in the list is preceded by a 2-octet header, with the first octet specifying the type of the parameter, and the second octet specifying the length. If there are no parameters, the value of *PARM. LEN* is zero and the message ends with no further data.

Only one parameter type is defined in the original standard: type *1* is reserved for authentication. The authentication parameter begins with a header that identifies the type of authentication followed by data appropriate for the type. The motivation for making authentication a parameter arises from a desire to allow BGP peers to choose an authentication mechanism without making the choice part of the BGP standard.

When it accepts an incoming *OPEN* message, a machine speaking BGP responds by sending a *KEEPALIVE* message (discussed below). Each side must send an *OPEN* and receive a *KEEPALIVE* message before they can exchange routing information. Thus, a *KEEPALIVE* message functions as the acknowledgement for an *OPEN*.

15.13 BGP UPDATE Message

Once BGP peers have created a TCP connection, sent *OPEN* messages, and acknowledged them, the peers use *UPDATE* messages to advertise new destinations that are reachable or to withdraw previous advertisements when a destination has become unreachable. Figure 15.8 illustrates the format of *UPDATE* messages.

As the figure shows, each *UPDATE* message is divided into two parts: the first lists previously advertised destinations that are being withdrawn, and the second specifies new destinations being advertised. As usual, fields labeled *variable* do not have a fixed size; if the information is not needed for a particular *UPDATE*, the field can be omitted from the message. Field *WITHDRAWN LEN* is a 2-octet field that specifies the size of the *Withdrawn Destinations* field that follows. If no destinations are being withdrawn, *WITHDRAWN LEN* contains zero. Similarly, the *PATH LEN* field specifies the size of the *Path Attributes* that are associated with new destinations being advertised. If there are no new destinations, the *PATH LEN* field contains zero.

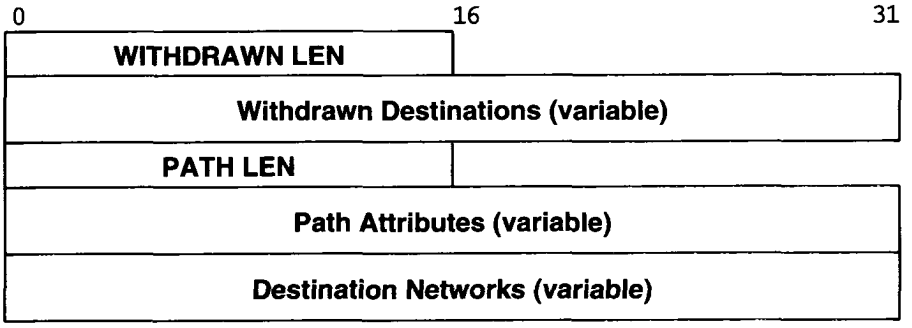


Figure 15.8 BGP UPDATE message format in which variable size areas of the message may be omitted. These octets follow the standard message header.

15.14 Compressed Mask-Address Pairs

Both the *Withdrawn Destinations* and the *Destination Networks* fields contain a list of IP network addresses. To accommodate classless addressing, BGP must send an address mask with each IP address. Instead of sending an address and a mask as separate 32-bit quantities, however, BGP uses a compressed representation to reduce message size. Figure 15.9 illustrates the format:

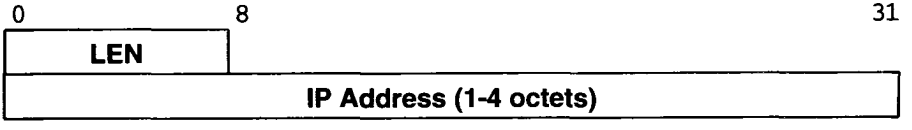


Figure 15.9 The compressed format BGP uses to store a destination address and the associated mask.

The figure shows that BGP does not actually send a bit mask. Instead, it encodes information about the mask into a single octet that precedes each address. The mask octet contains a binary integer that specifies the number of bits in the mask (mask bits are assumed to be contiguous). The address that follows the mask octet is also compressed — only those octets covered by the mask are included. Thus, only one address octet follows a mask value of *8* or less, two follow a mask value of *9* to *16*, three follow a mask value of *17* to *24*, and four follow a mask value of *25* to *32*. Interestingly, the standard also allows a mask octet to contain zero (in which case no address octets follow it). A zero length is useful because it corresponds to a default route.

15.15 BGP Path Attributes

We said that BGP is not a pure distance-vector protocol because it advertises more than a next hop. The additional information is contained in the *Path Attributes* field of an update message. A sender can use the path attributes to specify: a next hop for the advertised destinations, a list of autonomous systems along the path to the destinations, and whether the path information was learned from another autonomous system or derived from within the sender's autonomous system.

It is important to note that the path attributes are factored to reduce the size of the UPDATE message, meaning that the attributes apply to all destinations advertised in the message. Thus, if different attributes apply to some destinations, they must be advertised in a separate UPDATE message.

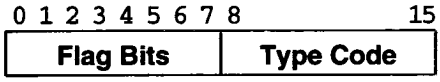
Path attributes are important in BGP for three reasons. First, path information allows a receiver to check for routing loops. The sender can specify an exact path through all autonomous systems to the destination. If any autonomous system appears more than once on the list, there must be a routing loop. Second, path information allows a receiver to implement policy constraints. For example, a receiver can examine paths to verify that they do not pass through untrusted autonomous systems (e.g., a competitor's autonomous system). Third, path information allows a receiver to know the source of all routes. In addition to allowing a sender to specify whether the information came from inside its autonomous system or from another system, the path attributes allow the sender to declare whether the information was collected with an exterior gateway protocol such as BGP or an interior gateway protocol†. Thus, each receiver can decide whether to accept or reject routes that originate in autonomous systems beyond the peer's.

Conceptually, the *Path Attributes* field contains a list of items, where each item consists of a triple:

(type, length, value)

Instead of fixed-size fields, the designers chose a flexible encoding scheme that minimizes the space each item occupies. As specified in Figure 15.10, the type information always requires two octets, but other fields vary in size.

†The next chapter describes interior gateway protocols.



Flag Bits	Description
0	0 for required attribute, 1 if optional
1	1 for transitive, 0 for nontransitive
2	0 for complete, 1 for partial
3	0 if length field is one octet; 1 if two
5-7	unused (must be zero)

Figure 15.10 Bits of the 2-octet type field that appears before each BGP attribute path item and the meaning of each.

For each item in the *Path Attributes* list, a length field follows the 2-octet type field, and may be either one or two octets long. As the figure shows, flag bit *3* specifies the size of the length field. A receiver uses the type field to determine the size of the length field, and then uses the contents of the length field to determine the size of the value field.

Each item in the *Path attributes* field can have one of seven possible type codes. Figure 15.11 summarizes the possibilities.

Type Code	Meaning
1	Specify origin of the path information
2	List of autonomous systems on path to destination
3	Next hop to use for destination
4	Discriminator used for multiple AS exit points
5	Preference used within an autonomous system
6	Indication that routes have been aggregated
7	ID of autonomous system that aggregated routes

Figure 15.11 The BGP attribute type codes and the meaning of each.

15.16 BGP KEEPALIVE Message

Two BGP peers periodically exchange *KEEPALIVE* messages to test network connectivity and to verify that both peers continue to function. A *KEEPALIVE* message consists of the standard message header with no additional data. Thus, the total message size is *19* octets (the minimum BGP message size).

There are two reasons why BGP uses keepalive messages. First, periodic message exchange is needed because BGP uses TCP for transport, and TCP does not include a mechanism to continually test whether a connection endpoint is reachable. However,

TCP does report an error to an application if it cannot deliver data the application sends. Thus, as long as both sides periodically send a keepalive, they will know if the TCP connection fails. Second, keepalives conserve bandwidth compared to other messages. Many early routing protocols used periodic exchange of routing information to test connectivity. However, because routing information changes infrequently, the message content seldom changes. Furthermore, because routing messages are usually large, resending the same message wastes network bandwidth needlessly. To avoid the inefficiency, BGP separates the functionality of route update from connectivity testing, allowing BGP to send small *KEEPALIVE* messages frequently, and reserving larger *UPDATE* messages for situations when reachability information changes.

Recall that a BGP speaker specifies a *hold timer* when it opens a connection; the hold timer specifies a maximum time that BGP is to wait without receiving a message. As a special case, the hold timer can be zero to specify that no *KEEPALIVE* messages are used. If the hold timer is greater than zero, the standard recommends setting the *KEEPALIVE* interval to one third of the hold timer. In no case can a BGP speaker make the *KEEPALIVE* interval less than one second (which agrees with the requirement that a nonzero hold timer cannot be less than three seconds).

15.17 Information From The Receiver's Perspective

Unlike most protocols that propagate routing information, an Exterior Gateway Protocol does not merely report the set of destinations it can reach. Instead, exterior protocols must provide information that is correct from the outsider's perspective. There are two issues: policies and optimal routes. The policy issue is obvious: a router inside an autonomous system may be allowed to reach a given destination, while outsiders are prohibited from reaching the same destination. The routing issue means that a router must advertise a next hop that is optimal from the outsider's perspective. Figure 15.12 illustrates the idea.

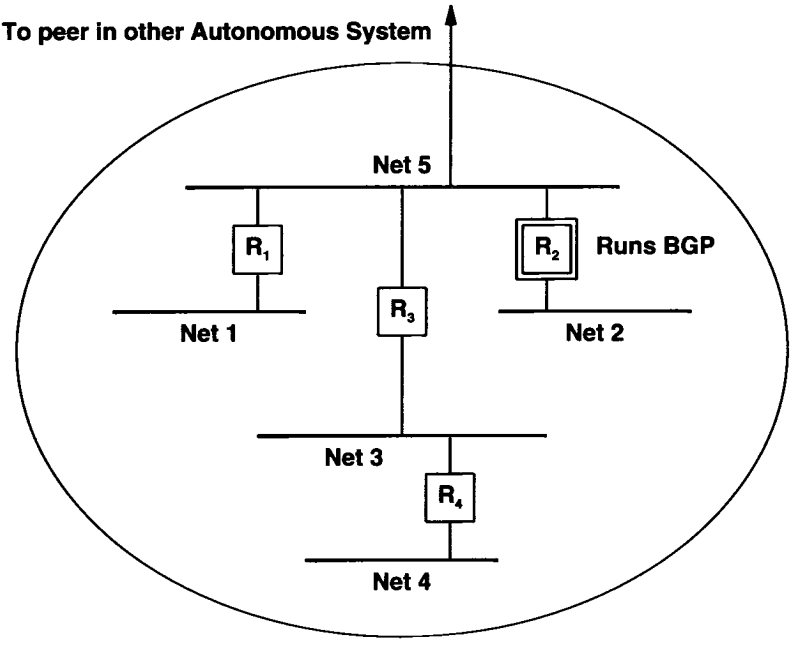


Figure 15.12 Example of an autonomous system. Router R_2 runs BGP and reports information from the outsider's perspective, not from its own routing table.

In the figure, router R_2 has been designated to speak BGP on behalf of the autonomous system. It must report reachability to networks *1* through *4*. However, when giving a next hop, it reports network *1* as reachable through router R_1, networks *3* and *4* as reachable through router R_3, and network *2* as reachable through R_2.

15.18 The Key Restriction Of Exterior Gateway Protocols

We have already seen that because exterior protocols follow policy restrictions, the networks they advertise may be a subset of the networks they can reach. However, there is a more fundamental limitation imposed on exterior routing:

> *An exterior gateway protocol does not communicate or interpret distance metrics, even if metrics are available.*

Protocols like BGP do allow a speaker to declare that a destination has become unreachable or to give a list of autonomous systems on the path to the destination, but

they cannot transmit or compare the cost of two routes unless the routes come from within the same autonomous system. In essence, BGP can only specify whether a path exists to a given destination; it cannot transmit or compute the shorter of two paths.

We can see now why BGP is careful to label the origin of information it sends. The essential observation is this: when a router receives advertisements for a given destination from peers in two different autonomous systems, it cannot compare the costs. Thus, advertising reachability with BGP is equivalent to saying, "My autonomous system provides a path to this network." There is no way for the router to say, "My autonomous system provides a better path to this network than another autonomous system."

Looking at interpretation of distances allows us to realize that BGP cannot be used as a routing algorithm. In particular, even if a router learns about two paths to the same network, it cannot know which path is shorter because it cannot know the cost of routes across intermediate autonomous systems. For example, consider a router that uses BGP to communicate with two peers in autonomous systems *p* and *f*. If the peer in autonomous system *p* advertises a path to a given destination through autonomous systems *p*, *q*, and *r*, and the peer in *f* advertises a path to the same destination through autonomous systems *f* and *g*, the receiver has no way of comparing the lengths of the two paths. The path through three autonomous systems might involve one local area network in each system, while the path through two autonomous systems might require several hops in each. Because a receiver does not obtain full routing information, it cannot compare.

Because it does not include a distance metric, an autonomous system must be careful to advertise only routes that traffic should follow. Technically, we say that an Exterior Gateway Protocol is a *reachability protocol* rather than a routing protocol. We can summarize:

> *Because an Exterior Gateway Protocol like BGP only propagates reachability information, a receiver can implement policy constraints, but cannot choose a least cost route. A sender must only advertise paths that traffic should follow.*

The key point here is that any internet which uses BGP to provide exterior routing information must either rely on policies or assume that each autonomous system crossing is equally expensive. Although it may seem innocuous, the restriction has some surprising consequences:

1. Although BGP can advertise multiple paths to a given network, it does not provide for the simultaneous use of multiple paths. That is, at any given instant, all traffic routed from a computer in one autonomous system to a network in another will traverse one path, even if multiple physical connections are present. Also note that an outside autonomous system will only use one return path even if the

source system divides outgoing traffic among two or more paths. As a result, delay and throughput between a pair of machines can be asymmetric, making an internet difficult to monitor or debug.

2. BGP does not support load sharing on routers between arbitrary autonomous systems. If two autonomous systems have multiple routers connecting them, one would like to balance the traffic equally among all routers. BGP allows autonomous systems to divide the load by network (e.g., to partition themselves into multiple subsets and have multiple routers advertise partitions), but it does not support more general load sharing.

3. As a special case of point 2, BGP alone is inadequate for optimal routing in an architecture that has two or more wide area networks interconnected at multiple points. Instead, managers must manually configure which networks are advertised by each exterior router.

4. To have rationalized routing, all autonomous systems in an internet must agree on a consistent scheme for advertising reachability. That is, BGP alone will not guarantee global consistency.

15.19 The Internet Routing Arbiter System

For an internet to operate correctly, routing information must be globally consistent. Individual protocols such as BGP that handle the exchange between a pair of routers, do not guarantee global consistency. Thus, a mechanism is needed to rationalize routing information globally. In the original Internet routing architecture, the core system guaranteed globally consistent routing information because at any time the core had exactly one path to each destination. When the core system was removed, a new mechanism was created to rationalize routing information.

Known as the *routing arbiter* (*RA*) system, the new mechanism consists of a replicated, authenticated database of reachability information. Updates to the database are *authenticated* to prevent an arbitrary router from advertising a path to a given destination. In general, only an autonomous system that owns a given network is allowed to advertise reachability. The need for such authentication became obvious in the early core system, which allowed any router to advertise reachability to any network. On several occasions, routing errors occurred when a router inadvertently advertised incorrect reachability information. The core accepted the information and changed routes, causing some networks to become unreachable.

To understand how other routers access the routing arbiter database, consider the current Internet architecture. We said that major ISPs interconnect at Network Access Points (NAPs). Thus, in terms of routing, a NAP represents the boundary between multiple autonomous systems. Although it would be possible to use BGP among each pair of ISPs at the NAP, doing so is both inefficient and prone to inconsistencies. Instead, each NAP has a computer called a *route server* (*RS*) that maintains a copy of the rout-

ing arbiter database and runs BGP. Each ISP designates one of its routers near a NAP to be a BGP border router. The designated border router maintains a connection to the route server over which it uses BGP. The ISP advertises reachability to its networks and the networks of its customers, and learns about networks in other ISPs.

One of the chief advantages of using BGP for route server access lies in its ability to carry negative information as well as positive information. When a destination becomes unreachable, the ISP informs the route server, which then makes the information available to other ISPs. Spreading negative information reduces unnecessary traffic because datagrams to unreachable destinations can be discarded before they transit from one ISP to another†.

15.20 BGP NOTIFICATION Message

In addition to the OPEN and UPDATE message types described above, BGP supports a *NOTIFICATION* message type used for control or when an error occurs. Errors are permanent — once it detects a problem, BGP sends a notification message and then closes the TCP connection. Figure 15.13 illustrates the message format.

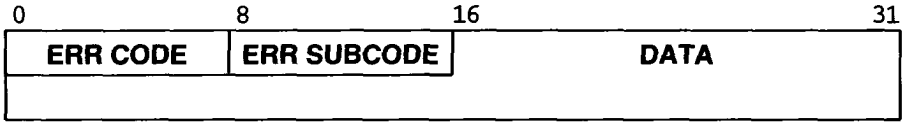


Figure 15.13 BGP NOTIFICATION message format. These octets follow the standard message header.

The 8-bit field labeled *ERR CODE* specifies one of the possible reasons listed in Figure 15.14.

ERR CODE	Meaning
1	Error in message header
2	Error in OPEN message
3	Error in UPDATE message
4	Hold timer expired
5	Finite state machine error
6	Cease (terminate connection)

Figure 15.14 The possible values of the *ERR CODE* field in a BGP NOTIFICATION message.

†Like the core system it replaced, the routing arbiter system does not include default routes. As a consequence, it is sometimes called a *default-free zone*.

For each possible *ERR CODE*, the *ERR SUBCODE* field contains a further explanation. Figure 15.15 lists the possible values.

Subcodes For Message Header Errors	
1	Connection not synchronized
2	Incorrect message length
3	Incorrect message type
Subcodes For OPEN Message Errors	
1	Version number unsupported
2	Peer AS invalid
3	BGP identifier invalid
4	Unsupported optional parameter
5	Authentication failure
6	Hold time unacceptable
Subcodes For UPDATE Message Errors	
1	Attribute list malformed
2	Unrecognized attribute
3	Missing attribute
4	Attribute flags error
5	Attribute length error
6	Invalid ORIGIN attribute
7	AS routing loop
8	Next hop invalid
9	Error in optional attribute
10	Invalid network field
11	Malformed AS path

Figure 15.15 The meaning of the *ERR SUBCODE* field in a BGP NOTIFICATION message.

15.21 Decentralization Of Internet Architecture

Two important architecture questions remain unanswered. The first focuses on centralization: how can the Internet architecture be modified to remove dependence on a (centralized) router system? The second concerns levels of trust: can an internet architecture be expanded to allow closer cooperation (trust) between some autonomous systems than among others?

Removing all dependence on a central system and adding trust are not easy. Although TCP/IP architectures continue to evolve, centralized roots are evident in many protocols. Without some centralization, each ISP would need to exchange reachability information with all ISPs to which it attached. Consequently, the volume of routing traffic would be significantly higher than with a routing arbiter scheme. Finally, centralization fills an important role in rationalizing routes and guaranteeing trust — in addition to storing the reachability database, the routing arbiter system guarantees global consistency and provides a trusted source of information.

15.22 Summary

Routers must be partitioned into groups or the volume of routing traffic would be intolerable. The connected Internet is composed of a set of autonomous systems, where each autonomous system consists of routers and networks under one administrative authority. An autonomous system uses an Exterior Gateway Protocol to advertise routes to other autonomous systems. Specifically, an autonomous system must advertise reachability of its networks to another system before its networks are reachable from sources within the other system.

The Border Gateway Protocol, BGP, is the most widely used Exterior Gateway Protocol. We saw that BGP contains three message types that are used to initiate communication (OPEN), send reachability information (UPDATE) and report an error condition (NOTIFICATION). Each message starts with a standard header that includes (optional) authentication information. BGP uses TCP for communication, but has a keepalive mechanism to ensure that peers remain in communication.

In the global Internet, each ISP is assigned to a separate autonomous system, and the main boundary among autonomous systems occurs at NAPs, where multiple ISPs interconnect. Instead of requiring pairs of ISPs to use BGP to exchange routing information, each NAP includes a route server. An ISP uses BGP to communicate with the route server, both to advertise reachability to its networks and its customers' networks as well as to learn about networks in other ISPs.

FOR FURTHER STUDY

Background on early Internet routing can be found in [RFCs 827, 888, 904, and 975]. Rekhter and Li [RFC 1771] describes version 4 of the Border Gateway Protocol *(BGP-4)*. BGP has been through three substantial revisions; earlier versions appear in [RFCs 1163, 1267, and 1654]. Traina [RFC 1773] reports experience with BGP-4, and Traina [RFC 1774] analyzes the volume of routing traffic generated. Finally, Villamizar et. al. {RFC 2439] considers the problem of route flapping.

EXERCISES

15.1 If your site runs an Exterior Gateway Protocol such as BGP, how many routes does NSFNET advertise?

15.2 Some implementations of BGP use a "hold down" mechanism that causes the protocol to delay accepting an *OPEN* from a peer for a fixed time following the receipt of a *cease request* message from that neighbor. Find out what problem a hold down helps solve.

15.3 For the networks in Figure 15.2, which router(s) should run BGP? Why?

15.4 The formal specification of BGP includes a finite state machine that explains how BGP operates. Draw a diagram of the state machine and label transitions.

15.5 What happens if a router in an autonomous system sends BGP routing update messages to a router in another autonomous system, claiming to have reachability for every possible internet destination?

15.6 Can two autonomous systems establish a routing loop by sending BGP update messages to one another? Why or why not?

15.7 Should a router that uses BGP to advertise routes treat the set of routes advertised differently than the set of routes in the local routing table? For example, should a router ever advertise reachability if it has not installed a route to that network in its routing table? Why or why not? Hint: read the RFC.

15.8 With regard to the previous question, examine the BGP-4 specification carefully. Is it legal to advertise reachability to a destination that is not listed in the local routing table?

15.9 If you work for a large corporation, find out whether it includes more than one autonomous system. If so, how do they exchange routing information?

15.10 What is the chief advantage of dividing a large, multi-national corporation into multiple autonomous systems? What is the chief disadvantage?

15.11 Corporations *A* and *B* use BGP to exchange routing information. To keep computers in *B* from reaching machines on one of its networks, *N*, the network administrator at corporation *A* configures BGP to omit *N* from advertisements sent to *B*. Is network *N* secure? Why or why not?

15.12 Because BGP uses a reliable transport protocol, KEEPALIVE messages cannot be lost. Does it make sense to specify a keepalive interval as one-third of the hold timer value? Why or why not?

15.13 Consult the RFCs for details of the *Path Attributes* field. What is the minimum size of a BGP UPDATE message?

16

Routing: In An Autonomous System (RIP, OSPF, HELLO)

16.1 Introduction

The previous chapter introduces the autonomous system concept and examines BGP, an Exterior Gateway Protocol that a router uses to advertise networks within its system to other autonomous systems. This chapter completes our overview of internet routing by examining how a router in an autonomous system learns about other networks within its autonomous system.

16.2 Static Vs. Dynamic Interior Routes

Two routers within an autonomous system are said to be *interior* to one another. For example, two routers on a university campus are considered interior to one another as long as machines on the campus are collected into a single autonomous system.

How can routers in an autonomous system learn about networks within the autonomous system? In small, slowly changing internets, managers can establish and modify routes by hand. The administrator keeps a table of networks and updates the table whenever a new network is added to, or deleted from, the autonomous system. For example, consider the small corporate internet shown in Figure 16.1.

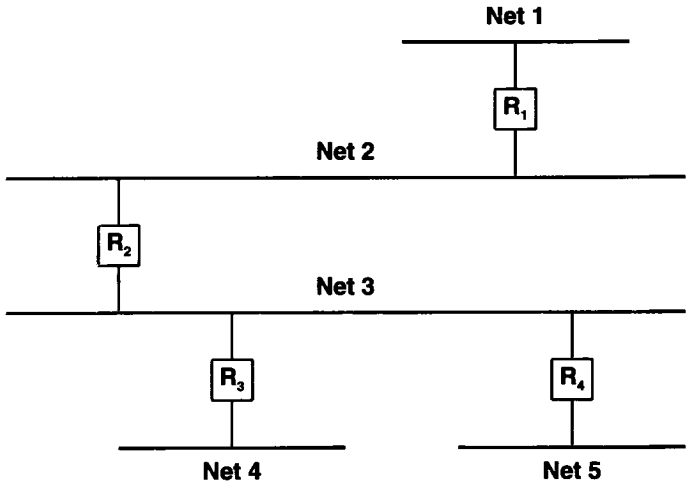


Figure 16.1 An example of a small internet consisting of 5 Ethernets and 4 routers at a single site. Only one possible route exists between any two hosts in this internet.

Routing for the internet in the figure is trivial because only one path exists between any two points. The manager can manually configure routes in all hosts and routers. If the internet changes (e.g., a new network is added), the manager must reconfigure the routes in all machines.

The disadvantages of a manual system are obvious: manual systems cannot accommodate rapid growth or rapid change. In large, rapidly changing environments like the global Internet, humans simply cannot respond to changes fast enough to handle problems; automated methods must be used. Automated methods can also help improve reliability and response to failure in small internets that have alternate routes. To see how, consider what happens if we add one additional router to the internet in Figure 16.1, producing the internet shown in Figure 16.2.

In internet architectures that have multiple physical paths, managers usually choose one to be the primary path. If the routers along the primary path fail, routes must be changed to send traffic along an alternate path. Changing routes manually is both time consuming and error-prone. Thus, even in small internets, an automated system should be used to change routes quickly and reliably.

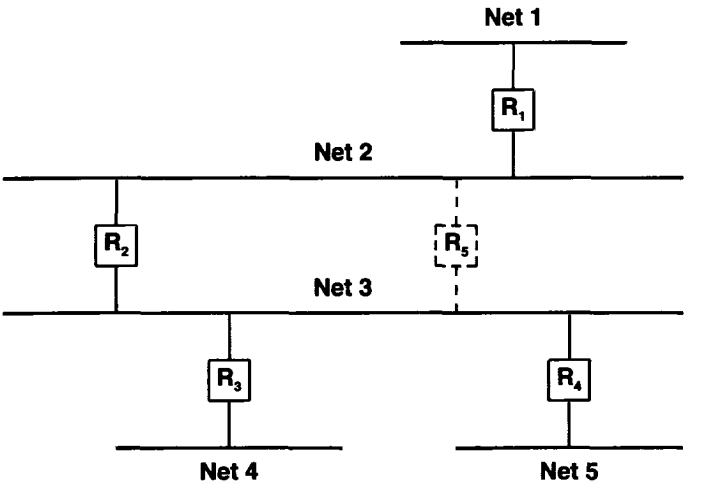


Figure 16.2 The addition of router R_5 introduces an alternate path between networks *2* and *3*. Routing software can quickly adapt to a failure and automatically switch routes to the alternate path.

To automate the task of keeping network reachability information accurate, interior routers usually communicate with one another, exchanging either network reachability data or network routing information from which reachability can be deduced. Once the reachability information for an entire autonomous system has been assembled, one of the routers in the system can advertise it to other autonomous systems using an Exterior Gateway Protocol.

Unlike exterior router communication, for which BGP provides a widely accepted standard, no single protocol has emerged for use within an autonomous system. Part of the reason for diversity comes from the varied topologies and technologies used in autonomous systems. Another part of the reason stems from the tradeoffs between simplicity and functionality — protocols that are easy to install and configure do not provide sophisticated functionality. As a result, a handful of protocols have become popular. Most small autonomous systems choose a single protocol, and use it exclusively to propagate routing information internally. Larger autonomous systems often choose a small set.

Because there is no single standard, we use the term *Interior Gateway Protocol* (*IGP*) as a generic description that refers to any algorithm that interior routers use when they exchange network reachability and routing information. For example, the last generation of core routers used a protocol named *SPREAD* as its Interior Gateway Protocol. Some autonomous systems use BGP as their IGP, although this seldom makes sense for small autonomous systems that span local area networks with broadcast capability.

Figure 16.3 illustrates two autonomous systems, each using an IGP to propagate routing information among its interior routers.

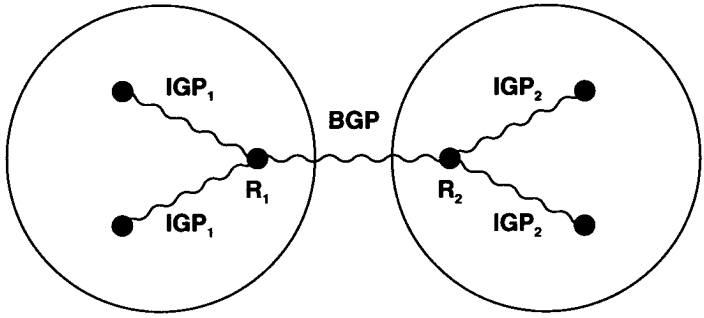


Figure 16.3 Conceptual view of two autonomous systems each using its own IGP internally, but using BGP to communicate between an exterior router and the other system.

In the figure, IGP_1 refers to the interior router protocol used within autonomous system *1*, and IGP_2 refers to the protocol used within autonomous system *2*. The figure also illustrates an important idea:

> *A single router may use two different routing protocols simultaneously, one for communication outside its autonomous system and another for communication within its autonomous system.*

In particular, routers that run BGP to advertise reachability usually also need to run an IGP to obtain information from within their autonomous system.

16.3 Routing Information Protocol (RIP)

16.3.1 History of RIP

One of the most widely used IGPs is the *Routing Information Protocol* (*RIP*), also known by the name of a program that implements it, *routed*†. The *routed* software was originally designed at the University of California at Berkeley to provide consistent routing and reachability information among machines on their local networks. It relies on physical network broadcast to make routing exchanges quickly. It was not designed to be used on large, wide area networks (although vendors now sell versions of RIP adapted for use on WANs).

Based on earlier internetworking research done at Xerox Corporation's Palo Alto Research Center (PARC), *routed* implements a protocol derived from the Xerox *NS Routing Information Protocol* (*RIP*), but generalizes it to cover multiple families of networks.

†The name comes from the UNIX convention of attaching "d" to the names of daemon processes; it is

Despite minor improvements over its predecessors, the popularity of RIP as an IGP does not arise from its technical merits alone. Instead, it is the result of Berkeley distributing *routed* software along with their popular 4BSD UNIX systems. Thus, many TCP/IP sites adopted and installed *routed*, and started using RIP without even considering its technical merits or limitations. Once installed and running, it became the basis for local routing, and research groups adopted it for larger networks.

Perhaps the most startling fact about RIP is that it was built and widely adopted before a formal standard was written. Most implementations were derived from the Berkeley code, with interoperability among them limited by the programmer's understanding of undocumented details and subtleties. As new versions appeared, more problems arose. An RFC standard appeared in June 1988, and made it possible for vendors to ensure interoperability.

16.3.2 RIP Operation

The underlying RIP protocol is a straightforward implementation of distance-vector routing for local networks. It partitions participants into *active* and *passive* (i.e., *silent*) machines. Active participants advertise their routes to others; passive participants listen to RIP messages and use them to update their routing table, but do not advertise. Only a router can run RIP in active mode; a host must use passive mode.

A router running RIP in active mode broadcasts a routing update message every 30 seconds. The update contains information taken from the router's current routing database. Each update contains a set of pairs, where each pair contains an IP network address and an integer distance to that network. RIP uses a *hop count metric* to measure distances. In the RIP metric, a router is defined to be one hop from a directly connected network†, two hops from a network that is reachable through one other router, and so on. Thus, the *number of hops* or the *hop count* along a path from a given source to a given destination refers to the number of routers that a datagram encounters along that path. It should be obvious that using hop counts to calculate shortest paths does not always produce optimal results. For example, a path with hop count *3* that crosses three Ethernets may be substantially faster than a path with hop count *2* that crosses two satellite connections. To compensate for differences in technologies, many RIP implementations allow managers to configure artificially high hop counts when advertising connections to slow networks.

Both active and passive RIP participants listen to all broadcast messages, and update their tables according to the distance-vector algorithm described earlier. For example, in the internet of Figure 16.2, router R_1 will broadcast a message on network *2* that contains the pair $(1, 1)$, meaning that it can reach network *1* at cost *1*. Routers R_2 and R_5 will receive the broadcast and install a route to network *1* through R_1 (at cost *2*). Later, routers R_2 and R_5 will include the pair $(1, 2)$ when they broadcast their RIP messages on network *3*. Eventually, all routers and hosts will install a route to network *1*.

RIP specifies a few rules to improve performance and reliability. For example, once a router learns a route from another router, it must apply *hysteresis*, meaning that it does not replace the route with an equal cost route. In our example, if routers R_2 and

†Other routing protocols define a direct connection to be zero hops.

R_5 both advertise network *1* at cost *2*, routers R_3 and R_4 will install a route through the one that happens to advertise first. We can summarize:

> *To prevent oscillation among equal cost paths, RIP specifies that existing routes should be retained until a new route has strictly lower cost.*

What happens if the first router to advertise a route fails (e.g., if it crashes)? RIP specifies that all listeners must timeout routes they learn via RIP. When a router installs a route in its table, it starts a timer for that route. The timer must be restarted whenever the router receives another RIP message advertising the route. The route becomes invalid if 180 seconds pass without the route being advertised again.

RIP must handle three kinds of errors caused by the underlying algorithm. First, because the algorithm does not explicitly detect routing loops, RIP must either assume participants can be trusted or take precautions to prevent such loops. Second, to prevent instabilities RIP must use a low value for the maximum possible distance (RIP uses *16*). Thus, for internets in which legitimate hop counts approach *16*, managers must divide the internet into sections or use an alternative protocol. Third, the distance-vector algorithm used by RIP can create a *slow convergence* or *count to infinity* problem, in which inconsistencies arise because routing update messages propagate slowly across the network. Choosing a small infinity (*16*) helps limit slow convergence, but does not eliminate it.

Routing table inconsistency is not unique to RIP. It is a fundamental problem that occurs with any distance-vector protocol in which update messages carry only pairs of destination network and distance to that network. To understand the problem consider the set of routers shown in Figure 16.4. The figure depicts routes to network *1* for the internet shown in Figure 16.2.

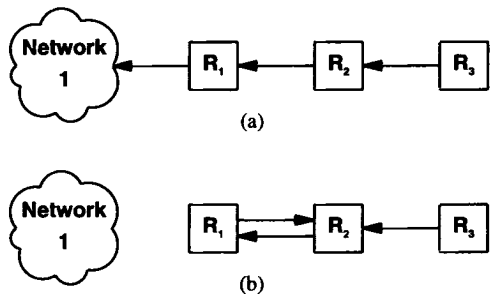


Figure 16.4 The slow convergence problem. In (a) three routers each have a route to network *1*. In (b) the connection to network *1* has vanished, but R_2 causes a loop by advertising it.

As Figure 16.4a shows, router R_1 has a direct connection to network *1*, so there is a route in its table with distance *1*, which will be included in its periodic broadcasts. Router R_2 has learned the route from R_1, installed the route in its routing table, and advertises the route at distance *2*. Finally, R_3 has learned the route from R_2 and advertises it at distance *3*.

Now suppose that R_1's connection to network *1* fails. R_1 will update its routing table immediately to make the distance *16* (infinity). In the next broadcast, R_1 will report the higher cost route. However, unless the protocol includes extra mechanisms to prevent it, some other router could broadcast its routes before R_1. In particular, suppose R_2 happens to advertise routes just after R_1*'s* connection fails. If so, R_1 will receive R_2's message and follow the usual distance-vector algorithm: it notices that R_2 has advertised a route to network *1* at lower cost, calculates that it now takes *3* hops to reach network *1* (*2* for R_2 to reach network *1* plus *1* to reach R_2*), and installs a new route with* R_2 listed as the next hop. Figure 16.4b depicts the result. At this point, if either R_1 or R_2 receives a datagram destined for network *1*, they will route the datagram back and forth until the datagram's time-to-live counter expires.

Subsequent RIP broadcasts by the two routers do not solve the problem quickly. In the next round of routing exchanges, R_1 broadcasts its routing table entries. When it learns that R_1's route to network *1* has distance *3*, R_2 calculates a new distance for its route, making it *4*. In the third round, R_1 receives a report from R_2 which includes the increased distance, and then increases the distance in its table to *5*. The two routers continue counting to RIP infinity.

16.3.3 Solving The Slow Convergence Problem

For the example in Figure 16.4, it is possible to solve the slow convergence problem by using a technique known as *split horizon update*. When using split horizon, a router does not propagate information about a route back over the same interface from which the route arrived. In the example, split horizon prevents router R_2 from advertising a route to network *1* back to router R_1, so if R_1 loses connectivity to network *1*, it must stop advertising a route. With split horizon, no routing loop appears in the example network. Instead, after a few rounds of routing updates, all routers will agree that the network is unreachable. However, the split horizon heuristic does not prevent routing loops in all possible topologies as one of the exercises suggests.

Another way to think of the slow convergence problem is in terms of information flow. If a router advertises a short route to some network, all receiving routers respond quickly to install that route. If a router stops advertising a route, the protocol must depend on a timeout mechanism before it considers the route unreachable. Once the timeout occurs, the router finds an alternative route and starts propagating that information. Unfortunately, a router cannot know if the alternate route depended on the route that just disappeared. Thus, negative information does not always propagate quickly. A short epigram captures the idea and explains the phenomenon:

Good news travels quickly; bad news travels slowly.

Another technique used to solve the slow convergence problem employs *hold down*. Hold down forces a participating router to ignore information about a network for a fixed period of time following receipt of a message that claims the network is unreachable. Typically, the hold down period is set to 60 seconds. The idea is to wait long enough to ensure that all machines receive the bad news and not mistakenly accept a message that is out of date. It should be noted that all machines participating in a RIP exchange need to use identical notions of hold down, or routing loops can occur. The disadvantage of a hold down technique is that if routing loops occur, they will be preserved for the duration of the hold down period. More important, the hold down technique preserves all incorrect routes during the hold down period, even when alternatives exist.

A final technique for solving the slow convergence problem is called *poison reverse*. Once a connection disappears, the router advertising the connection retains the entry for several update periods, and includes an infinite cost in its broadcasts. To make poison reverse most effective, it must be combined with *triggered updates*. Triggered updates force a router to send an immediate broadcast when receiving bad news, instead of waiting for the next periodic broadcast. By sending an update immediately, a router minimizes the time it is vulnerable to believing good news.

Unfortunately, while triggered updates, poison reverse, hold down, and split horizon techniques all solve some problems, they introduce others. For example, consider what happens with triggered updates when many routers share a common network. A single broadcast may change all their routing tables, triggering a new round of broadcasts. If the second round of broadcasts changes tables, it will trigger even more broadcasts. A broadcast avalanche can result†.

The use of broadcast, potential for routing loops, and use of hold down to prevent slow convergence can make RIP extremely inefficient in a wide area network. Broadcasting always takes substantial bandwidth. Even if no avalanche problems occur, having all machines broadcast periodically means that the traffic increases as the number of routers increases. The potential for routing loops can also be deadly when line capacity is limited. Once lines become saturated by looping packets, it may be difficult or impossible for routers to exchange the routing messages needed to break the loops. Also, in a wide area network, hold down periods are so long that the timers used by higher level protocols can expire and lead to broken connections. Despite these well-known problems, many groups continue to use RIP as an IGP in wide area networks.

16.3.4 RIP1 Message Format

RIP messages can be broadly classified into two types: routing information messages and messages used to request information. Both use the same format which consists of a fixed header followed by an optional list of network and distance pairs. Figure 16.5 shows the message format used with version *1* of the protocol, which is known as *RIP1*:

†To help avoid collisions on the underlying network, RIP requires each router to wait a small random time before sending a triggered update.

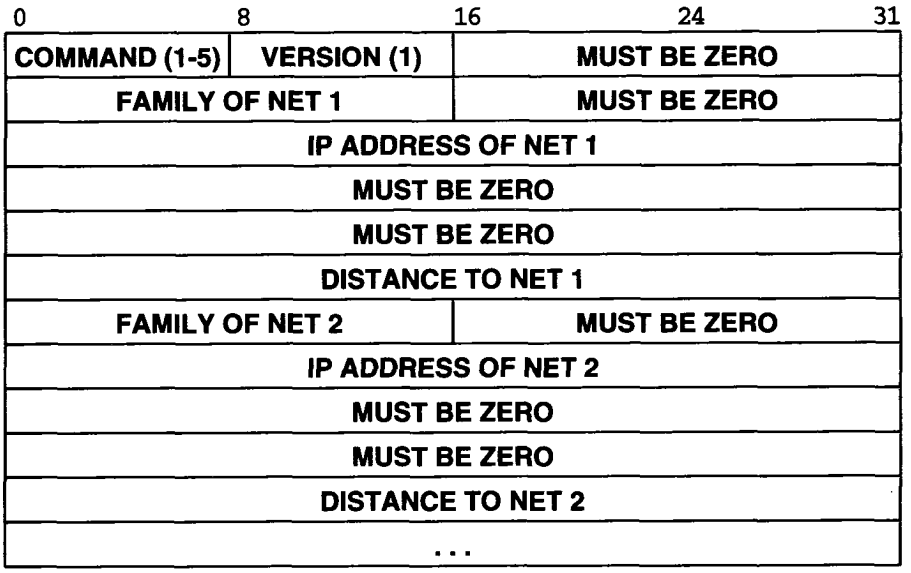


Figure 16.5 The format of a version 1 RIP message. After the 32-bit header, the message contains a sequence of pairs, where each pair consists of a network IP address and an integer distance to that network.

In the figure, field *COMMAND* specifies an operation according to the following table:

Command	Meaning
1	Request for partial or full routing information
2	Response containing network-distance pairs from sender's routing table
3	Turn on trace mode (obsolete)
4	Turn off trace mode (obsolete)
5	Reserved for Sun Microsystems internal use
9	Update Request (used with demand circuits)
10	Update Response (used with demand circuits)
11	Update Acknowledge (used with demand circuits)

A router or host can ask another router for routing information by sending a *request* command. Routers reply to requests using the *response* command. In most cases, however, routers broadcast unsolicited response messages periodically. Field *VERSION* contains the protocol version number (*1* in this case), and is used by the receiver to verify it will interpret the message correctly.

16.3.5 RIP1 Address Conventions

The generality of RIP is also evident in the way it transmits network addresses. The address format is not limited to use by TCP/IP; it can be used with multiple network protocol suites. As Figure 16.5 shows, each network address reported by RIP can have an address of up to 14 octets. Of course, IP addresses need only 4; RIP specifies that the remaining octets must be zero†. The field labeled *FAMILY OF NET i* identifies the protocol family under which the network address should be interpreted. RIP uses values assigned to address families under the 4BSD UNIX operating system (IP addresses are assigned value *2*).

In addition to normal IP addresses, RIP uses the convention that address *0.0.0.0* denotes a *default route*. RIP attaches a distance metric to every route it advertises, including default routes. Thus, it is possible to arrange for two routers to advertise a default route (e.g., a route to the rest of the internet) at different metrics, making one of them a primary path and the other a backup.

The final field of each entry in a RIP message, *DISTANCE TO NET i*, contains an integer count of the distance to the specified network. Distances are measured in router hops, but values are limited to the range *1* through *16*, with distance *16* used to signify infinity (i.e., no route exists).

16.3.6 RIP1 Route Interpretation And Aggregation

Because RIP was originally designed to be used with classful addresses, version 1 did not include any provision for a subnet mask. When subnet addressing was added to IP, version 1 of RIP was extended to permit routers to exchange subnetted addresses. However, because RIP1 update messages do not contain explicit mask information, an important restriction was added: a router can include host-specific or subnet-specific addresses in routing updates as long as all receivers can unambiguously interpret the addresses. In particular, subnet routes can only be included in updates sent across a network that is part of the subnetted prefix, and only if the subnet mask used with the network is the same as the subnet mask used with the address. In essence, the restriction means that RIP1 cannot be used to propagate variable-length subnet address or classless addresses. We can summarize:

> *Because it does not include explicit subnet information, RIP1 only permits a router to send subnet routes if receivers can unambiguously interpret the addresses according to the subnet mask they have available locally. As a consequence, RIP1 can only be used with classful or fixed-length subnet addresses.*

What happens when a router running RIP1 connects to one or more networks that are subnets of a prefix *N* as well as to one or more networks that are not part of *N*? The router must prepare different update messages for the two types of interfaces. Updates sent over the interfaces that are subnets of *N* can include subnet routes, but updates sent

†The designers chose to locate an IP address in the third through sixth octets of the address field to ensure 32-bit alignment.

over other interfaces cannot. Instead, when sending over other interfaces the router is required to *aggregate* the subnet information and advertise a single route to network *N*.

16.3.7 RIP2 Extensions

The restriction on address interpretation means that version 1 of RIP cannot be used to propagate either variable-length subnet addresses or the classless addresses used with CIDR. When version 2 of RIP (*RIP2*) was defined, the protocol was extended to include an explicit subnet mask along with each address. In addition, RIP2 updates include explicit next-hop information, which prevents routing loops and slow convergence. As a result, RIP2 offers significantly increased functionality as well as improved resistance to errors.

16.3.8 RIP2 Message Format

The message format used with RIP2 is an extension of the RIP1 format, with additional information occupying unused octets of the address field. In particular, each address includes an explicit next hop as well as an explicit subnet mask as Figure 16.6 illustrates.

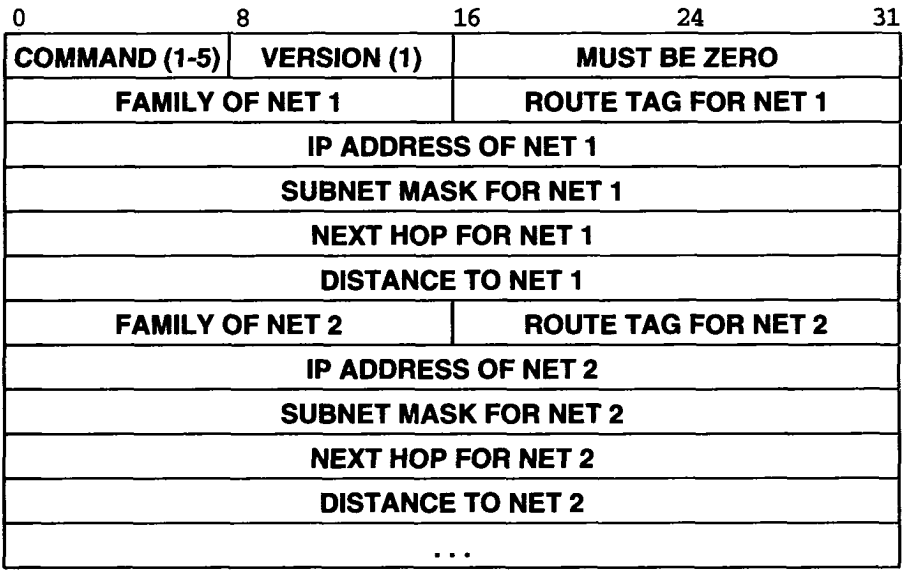


Figure 16.6 The format of a RIP2 message. In addition to pairs of a network IP address and an integer distance to that network, the message contains a subnet mask for each address and explicit next-hop information.

RIP2 also attaches a 16-bit *ROUTE TAG* field to each entry. A router must send the same tag it receives when it transmits the route. Thus, the tag provides a way to propagate additional information such as the origin of the route. In particular, if RIP2 learns a route from another autonomous system, it can use the *ROUTE TAG* to propagate the autonomous system's number.

Because the version number in RIP2 occupies the same octet as in RIP1, both versions of the protocols can be used on a given router simultaneously without interference. Before processing an incoming message, RIP software examines the version number.

16.3.9 Transmitting RIP Messages

RIP messages do not contain an explicit length field or an explicit count of entries. Instead, RIP assumes that the underlying delivery mechanism will tell the receiver the length of an incoming message. In particular, when used with TCP/IP, RIP messages rely on UDP to tell the receiver the message length. RIP operates on UDP port *520*. Although a RIP request can originate at other UDP ports, the destination UDP port for requests is always *520*, as is the source port from which RIP broadcast messages originate.

16.3.10 The Disadvantage Of RIP Hop Counts

Using RIP as an interior router protocol limits routing in two ways. First, RIP restricts routing to a hop-count metric. Second, because it uses a small value of hop count for infinity, RIP restricts the size of any internet using it. In particular, RIP restricts the *span* of an internet (i.e., the maximum distance across) to 16. That is, an internet using RIP can have at most 15 routers between any two hosts.

Note that the limit on network span is neither a limit on the total number of routers nor a limit on density. In fact, most campus networks have a small span even if they have many routers because the topology is arranged as a *hierarchy*. Consider, for example, a typical corporate intranet. Most use a hierarchy that consists of a high-speed backbone network with multiple routers each connecting the backbone to a workgroup, where each workgroup occupies a single LAN. Although the corporation can include dozens of workgroups, the span of the entire intranet is only 2. Even if each workgroup is extended to include a router that connects one or more additional LANs, the maximum span only increases to 4. Similarly, extending the hierarchy one more level only increases the span to 6. Thus, the limit that RIP imposes affects large autonomous systems or autonomous systems that do not have a hierarchical organization.

Even in the best cases, however, hop counts provide only a crude measure of network capacity or responsiveness. Thus, using hop counts does not always yield routes with least delay or highest capacity. Furthermore, computing routes on the basis of minimum hop counts has the severe disadvantage that it makes routing relatively static because routes cannot respond to changes in network load. The next sections consider an alternative metric, and explain why hop count metrics remain popular despite their limitations.

16.4 The Hello Protocol

The HELLO protocol provides an example of an IGP that uses a routing metric other than hop count. Although HELLO is now obsolete, it was significant in the history of the Internet because it was the IGP used among the original NSFNET backbone "fuzzball" routers†. HELLO is significant to us because it provides an example of a protocol that uses a metric of delay.

HELLO provides two functions: it synchronizes the clocks among a set of machines, and it allows each machine to compute shortest delay paths to destinations. Thus, HELLO messages carry timestamp information as well as routing information. The basic idea behind HELLO is simple: each machine participating in the HELLO exchange maintains a table of its best estimate of the clocks in neighboring machines. Before transmitting a packet, a machine adds its timestamp by copying the current clock value into the packet. When a packet arrives, the receiver computes an estimate of the current delay on the link by subtracting the timestamp on the incoming packet from the local estimate for the current clock in the neighbor. Periodically, machines poll their neighbors to reestablish estimates for clocks.

HELLO messages also allow participating machines to compute new routes. The protocol uses a modified distance-vector scheme that uses a metric of delay instead of hop count. Thus, each machine periodically sends its neighbors a table of destinations it can reach and an estimated delay for each. When a message arrives from machine *X*, the receiver examines each entry in the message and changes the next hop to *X* if the route through *X* is less expensive than the current route (i.e., any route where the delay to *X* plus the delay from *X* to the destination is less than the current delay to the destination).

16.5 Delay Metrics And Oscillation

It may seem that using delay as a routing metric would produce better routes than using a hop count. In fact, HELLO worked well in the early Internet backbone. However, there is an important reasons why delay is not used as a metric in most protocols: instability.

Even if two paths have identical characteristics, any protocol that changes routes quickly can become unstable. Instability arises because delay, unlike hop counts, is not fixed. Minor variations in delay measurements occur because of hardware clock drift, CPU load during measurement, or bit delays caused by link-level synchronization. Thus, if a routing protocol reacts quickly to slight differences in delay, it can produce a two-stage oscillation effect in which traffic switches back and forth between the alternate paths. In the first stage, the router finds the delay on path *1* slightly less and abruptly switches traffic onto it. In the next round, the router finds that path *B* has slightly less delay and switches traffic back.

To help avoid oscillation, protocols that use delay implement several heuristics. First, they employ the *hold down* technique discussed previously to prevent routes from

†The term *fuzzball* referred to a noncommercial router that consisted of specially-crafted protocol software running on a PDP11 computer.

changing rapidly. Second, instead of measuring as accurately as possible and comparing the values directly, the protocols round measurements to large multiples or implement a minimum *threshold* by ignoring differences less than the threshold. Third, instead of comparing each individual delay measurement, they keep a running *average* of recent values or alternatively apply a *K-out-of-N* rule that requires at least *K* of the most recent *N* delay measurements be less than the current delay before the route can be changed.

Even with heuristics, protocols that use delay can become unstable when comparing delays on paths that do not have identical characteristics. To undersand why, it is necessary to know that traffic can have a dramatic effect on delay. With no traffic, the network delay is simply the time required for the hardware to transfer bits from one point to another. As the traffic load imposed on the network increases, however, delays begin to rise because routers in the system need to enqueue packets that are waiting for transmission. If the load is even slightly more than 100% of the network capacity, the queue becomes unbounded, meaning that the effective delay becomes infinite. To summarize:

> *The effective delay across a network depends on traffic; as the load increases to 100% of the network capacity, delay grows rapidly.*

Because delays are extremely sensitive to changes in load, protocols that use delay as a metric can easily fall into a *positive feedback cycle*. The cycle is triggered by a small external change in load (e.g., one computer injecting a burst of additional traffic). The increased traffic raises the delay, which causes the protocol to change routes. However, because a route change affects the load, it can produce an even larger change in delays, which means the protocol will again recompute routes. As a result, protocols that use delay must contain mechanisms to dampen oscillation.

We described heuristics that can solve simple cases of route oscillation when paths have identical throughput characteristics and the load is not excessive. The heuristics can become ineffective, however, when alternative paths have different delay and throughput characteristics. As an example consider the delay on two paths: one over a satellite and the other over a low capacity serial line (e.g., a 9600 baud serial line). In the first stage of the protocol when both paths are idle, the serial line will appear to have significantly lower delay than the satellite, and will be chosen for traffic. Because the serial line has low capacity, it will quickly become overloaded, and the delay will rise sharply. In the second stage, the delay on the serial line will be much greater than that of the satellite, so the protocol will switch traffic away from the overloaded path. Because the satellite path has large capacity, traffic which overloaded the serial line does not impose a significant load on the satellite, meaning that the delay on the satellite path does not change with traffic. In the next round, the delay on the unloaded serial line will once again appear to be much smaller than the delay on the satellite path. The protocol will reverse the routing, and the cycle will continue. Such oscillations do, in fact, occur in practice. As the example shows, they are difficult to manage because traffic which has little effect on one network can overload another.

16.6 Combining RIP, Hello, And BGP

We have already observed that a single router may use both an Interior Gateway Protocol to gather routing information within its autonomous system and an Exterior Gateway Protocol to advertise routes to other autonomous systems. In principle, it should be easy to construct a single piece of software that combines the two protocols, making it possible to gather routes and advertise them without human intervention. In practice, technical and political obstacles make doing so complex.

Technically, IGP protocols, like RIP and Hello, are routing protocols. A router uses such protocols to update its routing table based on information it acquires from other routers inside its autonomous system. Thus, *routed*, the UNIX program that implements RIP, advertises information from the local routing table and changes the local routing table when it receives updates. RIP trusts routers within the same autonomous system to pass correct data.

In contrast, exterior protocols such as BGP do not trust routers in other autonomous systems. Consequently, exterior protocols do not advertise all possible routes from the local routing table. Instead, such protocols keep a database of network reachability, and apply policy constraints when sending or receiving information. Ignoring such policy constraints can affect routing in a larger sense — some parts of the internet can be become unreachable. For example, if a router in an autonomous system that is running RIP happens to propagate a low-cost route to a network at Purdue University when it has no such route, other routers running RIP will accept and install the route. They will then pass Purdue traffic to the router that made the error. As a result, it may be impossible for hosts in that autonomous system to reach Purdue. The problem becomes more serious if Exterior Gateway Protocols do not implement policy constraints. For example, if a border router in the autonomous system uses BGP to propagate the illegal route to other autonomous systems, the network at Purdue may become unreachable from some parts of the internet.

16.7 Inter-Autonomous System Routing

We have seen that EGPs such as BGP allow one autonomous system to advertise reachability information to another. However, it would be useful to also provide *inter-autonomous system routing* in which routers choose least-cost paths. Doing so requires additional trust. Extending the notions of trust from a single autonomous system to multiple autonomous systems is complex. The simplest approach groups autonomous systems hierarchically. Imagine, for example, three autonomous systems in three separate academic departments on a large university campus. It is natural to group these three together because they share administrative ties. The motivation for hierarchical grouping comes primarily from the notion of trust. Routers within a group trust one another with a higher level of confidence than routers in separate groups.

Grouping autonomous systems requires extensions to routing protocols. When reporting distances, the values must be increased when passing across the boundary from

one group to another. The technique, loosely called *metric transformation*, partitions distance values into three categories. For example, suppose routers within an autonomous system use distance values less than 128. We can make a rule that when passing distance information across an autonomous system boundary within a single group, the distances must be transformed into the range of 128 to 191. Finally, we can make a rule that when passing distance values across the boundary between two groups, the values must be transformed into the range of 192 to 254†. The effect of such transformations is obvious: for any given destination network, any path that lies entirely within the autonomous system is guaranteed to have lower cost than a path that strays outside the autonomous system. Furthermore, among all paths that stray outside the autonomous system, those that remain within the group have lower cost than those that cross group boundaries. The key advantage of metric transformations is that they allow each autonomous system to choose an IGP, yet make it possible for other systems to compare routing costs.

16.8 Gated: Inter-Autonomous System Communication

A mechanism has been created to provide an interface between autonomous systems. Known as *gated*‡, the mechanism understands multiple protocols (both IGPs and BGP), and ensures that policy constraints are honored. For example, *gated* can accept RIP messages and modify the local computer's routing table just like the *routed* program. It can also advertise routes from within its autonomous system using BGP. The rules gated follows allow a system administrator to specify exactly which networks *gated* may and may not advertise and how to report distances to those networks. Thus, although *gated* is not an IGP, it plays an important role in routing because it demonstrates that it is feasible to build an automated mechanism linking an IGP with BGP without sacrificing protection.

Gated performs another useful task by implementing metric transformations. Thus, it is possible and convenient to use gated between two autonomous systems as well as on the boundary between two groups of routers that each participate in an IGP.

16.9 The Open SPF Protocol (OSPF)

In Chapter 14, we said that a link state routing algorithm, which uses SPF to compute shortest paths, scales better than a distance-vector algorithm. To encourage the adoption of link state technology, a working group of the Internet Engineering Task Force has designed an interior gateway protocol that uses the link state algorithm. Called *Open SPF (OSPF)*, the new protocol tackles several ambitious goals.

• As the name implies, the specification is available in the published literature. Making it an open standard that anyone can implement without paying license fees has encouraged many vendors to support OSPF. Consequently, it has become a popular replacement for proprietary protocols.

---

†The term *autonomous confederation* has been used to describe a group of autonomous systems; boundaries of autonomous confederations correspond to transformations beyond 191.

‡The name *gated* is pronounced "gate d" from "gate daemon."

• OSPF includes *type of service routing*. Managers can install multiple routes to a given destination, one for each priority or type of service. When routing a datagram, a router running OSPF uses both the destination address and type of service field in an IP header to choose a route. OSPF is among the first TCP/IP protocols to offer type of service routing.

• OSPF provides *load balancing*. If a manager specifies multiple routes to a given destination at the same cost, OSPF distributes traffic over all routes equally. Again, OSPF is among the first open IGPs to offer load balancing; protocols like RIP compute a single route to each destination.

• To permit growth and make the networks at a site easier to manage, OSPF allows a site to partition its networks and routers into subsets called *areas*. Each area is self-contained; knowledge of an area's topology remains hidden from other areas. Thus, multiple groups within a given site can cooperate in the use of OSPF for routing even though each group retains the ability to change its internal network topology independently.

• The OSPF protocol specifies that all exchanges between routers can be *authenticated*. OSPF allows a variety of authentication schemes, and even allows one area to choose a different scheme than another area. The idea behind authentication is to guarantee that only trusted routers propagate routing information. To understand why this could be a problem, consider what can happen when using RIP1, which has no authentication. If a malicious person uses a personal computer to propagate RIP messages advertising low-cost routes, other routers and hosts running RIP will change their routes and start sending datagrams to the personal computer.

• OSPF includes support for host-specific, subnet-specific, and classless routes as well as classful network-specific routes. All types may be needed in a large internet.

• To accommodate multi-access networks like Ethernet, OSPF extends the SPF algorithm described in Chapter 14. We described the algorithm using a point-to-point graph and said that each router running SPF would periodically broadcast link status messages about each reachable neighbor. If K routers attach to an Ethernet, they will broadcast K^2 reachability messages. OSPF minimizes broadcasts by allowing a more complex graph topology in which each node represents either a router or a network. Consequently, OSPF allows every multi-access network to have a *designated gateway* (i.e., a *designated router*) that sends link status messages on behalf of all routers attached to the network; the messages report the status of all links from the network to routers attached to the network. OSPF also uses hardware broadcast capabilities, where they exist, to deliver link status messages.

• To permit maximum flexibility, OSPF allows managers to describe a virtual network topology that abstracts away from the details of physical connections. For example, a manager can configure a virtual link between two routers in the routing graph even if the physical connection between the two routers requires communication across a transit network.

• OSPF allows routers to exchange routing information learned from other (external) sites. Basically, one or more routers with connections to other sites learn information about those sites and include it when sending update messages. The message for-

mat distinguishes between information acquired from external sources and information acquired from routers interior to the site, so there is no ambiguity about the source or reliability of routes.

16.9.1 OSPF Message Format

Each OSPF message begins with a fixed, 24-octet header as Figure 16.7 shows:

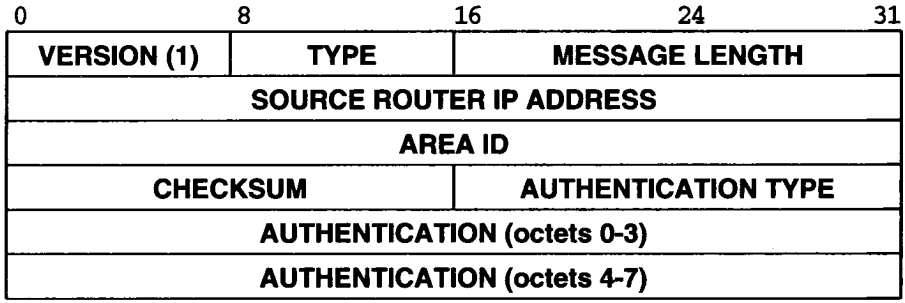


Figure 16.7 The fixed 24-octet OSPF message header.

Field *VERSION* specifies the version of the protocol. Field *TYPE* identifies the message type as one of:

Type	Meaning
1	Hello (used to test reachability)
2	Database description (topology)
3	Link status request
4	Link status update
5	Link status acknowledgement

The field labeled *SOURCE ROUTER IP ADDRESS* gives the address of the sender, and the field labeled *AREA ID* gives the 32-bit identification number for the area.

Because each message can include authentication, field *AUTHENTICATION TYPE* specifies which authentication scheme is used (currently, *0* means no authentication and *1* means a simple password is used).

16.9.2 OSPF Hello Message Format

OSPF sends *hello* messages on each link periodically to establish and test neighbor reachability. Figure 16.8 shows the format.

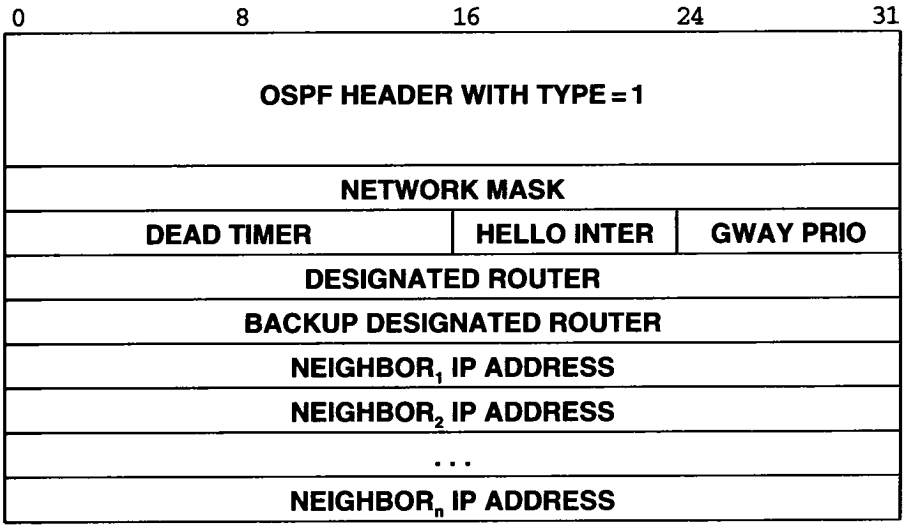


Figure 16.8 OSPF *hello* message format. A pair of neighbor routers exchanges these messages periodically to test reachability.

Field *NETWORK MASK* contains a mask for the network over which the message has been sent (see Chapter 10 for details about masks). Field *DEAD TIMER* gives a time in seconds after which a nonresponding neighbor is considered dead. Field *HELLO INTER* is the normal period, in seconds, between hello messages. Field *GWAY PRIO* is the integer priority of this router, and is used in selecting a backup designated router. The fields labeled *DESIGNATED ROUTER* and *BACKUP DESIGNATED ROUTER* contain IP addresses that give the sender's view of the designated router and backup designated router for the network over which the message is sent. Finally, fields labeled *NEIGHBOR$_i$ IP ADDRESS* give the IP addresses of all neighbors from which the sender has recently received hello messages.

16.9.3 OSPF Database Description Message Format

Routers exchange OSPF *database description* messages to initialize their network topology database. In the exchange, one router serves as a master, while the other is a slave. The slave acknowledges each database description message with a response. Figure 16.9 shows the format.

Because it can be large, the topology database may be divided into several messages using the *I* and *M* bits. Bit *I* is set to *1* in the initial message; bit *M* is set to *1* if additional messages follow. Bit *S* indicates whether a message was sent by a master (*1*) or by a slave (*0*). Field *DATABASE SEQUENCE NUMBER* numbers messages sequentially so the receiver can tell if one is missing. The initial message contains a random integer *R*; subsequent messages contain sequential integers starting at *R*.

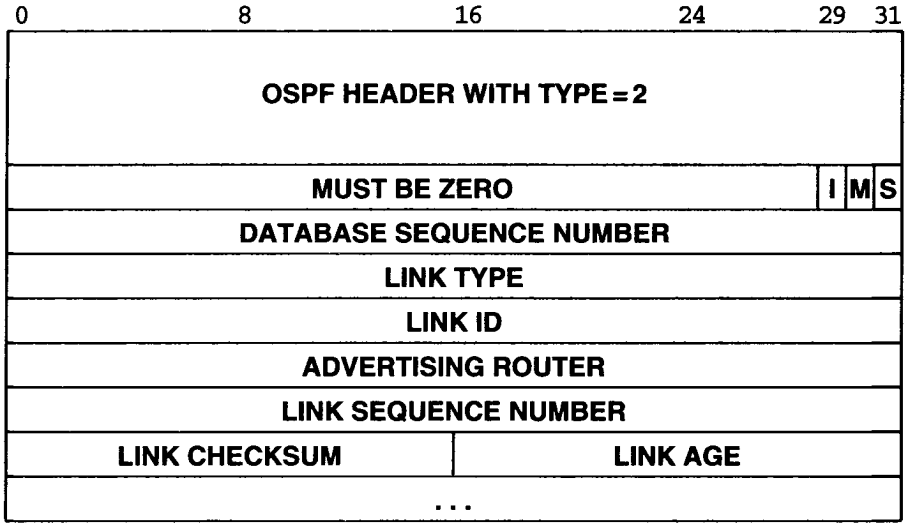


Figure 16.9 OSPF *database description* message format. The fields starting at *LINK TYPE* are repeated for each link being specified.

The fields from *LINK TYPE* through *LINK AGE* describe one link in the network topology; they are repeated for each link. The *LINK TYPE* describes a link according to the following table.

Link Type	Meaning
1	Router link
2	Network link
3	Summary link (IP network)
4	Summary link (link to border router)
5	External link (link to another site)

Field *LINK ID* gives an identification for the link (which can be the IP address of a router or a network, depending on the link type).

Field *ADVERTISING ROUTER* specifies the address of the router advertising this link, and *LINK SEQUENCE NUMBER* contains an integer generated by that router to ensure that messages are not missed or received out of order. Field *LINK CHECKSUM* provides further assurance that the link information has not been corrupted. Finally, field *LINK AGE* also helps order messages — it gives the time in seconds since the link was established.

16.9.4 OSPF Link Status Request Message Format

After exchanging database description messages with a neighbor, a router may discover that parts of its database are out of date. To request that the neighbor supply updated information, the router sends a *link status request* message. The message lists specific links as shown in Figure 16.10. The neighbor responds with the most current information it has about those links. The three fields shown are repeated for each link about which status is requested. More than one request message may be needed if the list of requests is long.

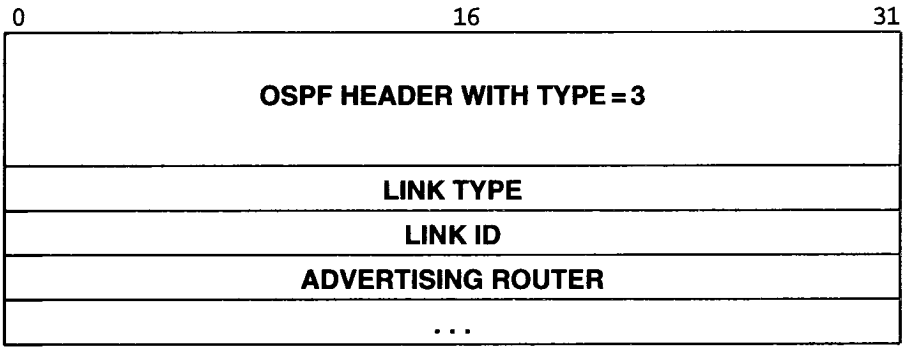


Figure 16.10 OSPF *link status request* message format. A router sends this message to a neighbor to request current information about a specific set of links.

16.9.5 OSPF Link Status Update Message Format

Routers broadcast the status of links with a *link status update* message. Each update consists of a list of advertisements, as Figure 16.11 shows.

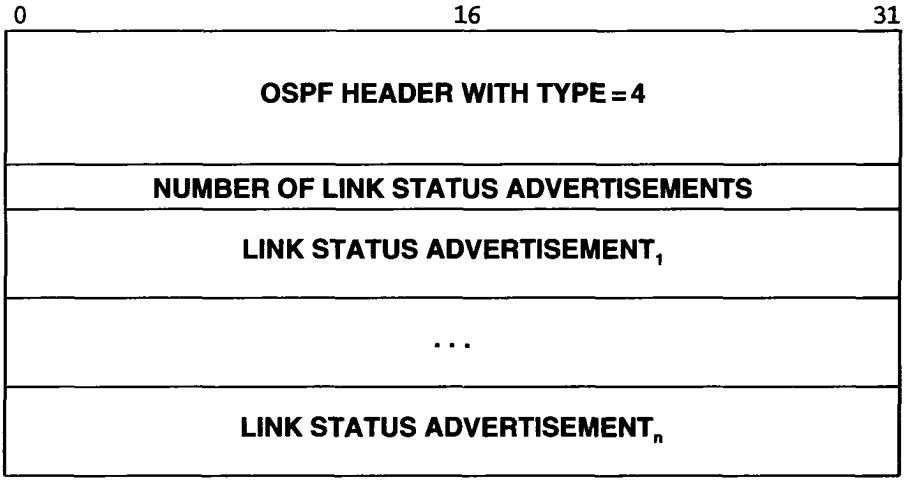


Figure 16.11 OSPF *link status update* message format. A router sends such a message to broadcast information about its directly connected links to all other routers.

Each link status advertisement has a header format as shown in Figure 16.12. The values used in each field are the same as in the database description message.

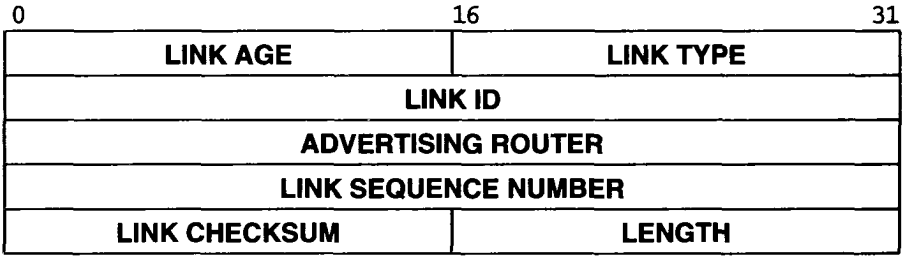


Figure 16.12 The format of the header used for all link status advertisements.

Following the link status header comes one of four possible formats to describe the links from a router to a given area, the links from a router to a specific network, the links from a router to the physical networks that comprise a single, subnetted IP network (see Chapter 10), or the links from a router to networks at other sites. In all cases, the *LINK TYPE* field in the link status header specifies which of the formats has been used. Thus, a router that receives a link status update message knows exactly which of the described destinations lie inside the site and which are external.

16.10 Routing With Partial Information

We began our discussion of internet router architecture and routing by discussing the concept of partial information. Hosts can route with only partial information because they rely on routers. It should be clear now that not all routers have complete information. Most autonomous systems have a single router that connects the autonomous system to other autonomous systems. For example, if the site connects to the global Internet, at least one router must have a connection that leads from the site to an ISP. Routers within the autonomous system know about destinations within that autonomous system, but they use a default route to send all other traffic to the ISP.

How to do routing with partial information becomes obvious if we examine a router's routing tables. Routers at the center of the Internet have a complete set of routes to all possible destinations that they learn from the routing arbiter system; such routers do not use default routing. In fact, if a destination network address does not appear in the routing arbiter database, only two possibilities exist: either the address is not a valid destination IP address, or the address is valid but currently unreachable (e.g., because routers or networks leading to that address have failed). Routers beyond those in ISPs at the center of the Internet do not usually have a complete set of routes; they rely on a default route to handle network addresses they do not understand.

Using default routes for most routers has two consequences. First, it means that local routing errors can go undetected. For example, if a machine in an autonomous system incorrectly routes a packet to an external autonomous system instead of to a local router, the external system will route it back (perhaps to a different entry point). Thus, connectivity may appear to be preserved even if routing is incorrect. The problem may not seem severe for small autonomous systems that have high speed local area networks, but in a wide area network, incorrect routes can be disastrous. Second, on the positive side, using default routes whenever possible means that the routing update messages exchanged by most routers will be much smaller than they would be if complete information had to be included.

16.11 Summary

Managers must choose how to pass routing information among the local routers within an autonomous system. Manual maintenance of routing information suffices only for small, slowly changing internets that have minimal interconnection; most require automated procedures that discover and update routes automatically. Two routers under the control of a single manager run an Interior Gateway Protocol, IGP, to exchange routing information.

An IGP implements either the distance-vector algorithm or the link state algorithm, which is known by the name Shortest Path First (SPF). We examined three specific IGPs: RIP, HELLO, and OSPF. RIP, a distance-vector protocol implemented by the UNIX program *routed*, is among the most popular. It uses split horizon, hold-down, and poison reverse techniques to help eliminate routing loops and the problem of count-

ing to infinity. Although it is obsolete, Hello is interesting because it illustrates a distance-vector protocol that uses delay instead of hop counts as a distance metric. We discussed the disadvantages of delay as a routing metric, and pointed out that although heuristics can prevent instabilities from arising when paths have equal throughput characteristics, long-term instabilities arise when paths have different characteristics. Finally, OSPF is a protocol that implements the link status algorithm.

Also, we saw that the *gated* program provides an interface between an Interior Gateway Protocol like RIP and the Exterior Gateway Protocol, BGP, automating the process of gathering routes from within an autonomous system and advertising them to another autonomous system.

FOR FURTHER STUDY

Hedrick [RFC 1058] discusses algorithms for exchanging routing information in general and contains the standard specification for RIP1. Malkin [RFC 2453] gives the standard for RIP2. The HELLO protocol is documented in Mills [RFC 891]. Mills and Braun [1987] considers the problems of converting between delay and hop-count metrics. Moy [RFC 1583] contains the lengthy specification of OSPF as well as a discussion of the motivation behind it. Fedor [June 1988] describes *gated.*

EXERCISES

16.1 What network families does RIP support? Hint: read the networking section of the 4.3 BSD UNIX Programmer's Manual.

16.2 Consider a large autonomous system using an interior router protocol like HELLO that bases routes on delay. What difficulty does this autonomous system have if a subgroup decides to use RIP on its routers?

16.3 Within a RIP message, each IP address is aligned on a 32-bit boundary. Will such addresses be aligned on a 32-bit boundary if the IP datagram carrying the message starts on a 32-bit boundary?

16.4 An autonomous system can be as small as a single local area network or as large as multiple long haul networks. Why does the variation in size make it difficult to find a standard IGP?

16.5 Characterize the circumstances under which the split horizon technique will prevent slow convergence.

16.6 Consider an internet composed of many local area networks running RIP as an IGP. Find an example that shows how a routing loop can result even if the code uses "hold down" after receiving information that a network is unreachable.

16.7 Should a host ever run RIP in active mode? Why or why not?

16.8 Under what circumstances will a hop count metric produce better routes than a metric that uses delay?

16.9 Can you imagine a situation in which an autonomous system chooses *not* to advertise all its networks? Hint: think of a university.

16.10 In broad terms, we could say that RIP distributes the local routing table, while BGP distributes a table of networks and routers used to reach them (i.e., a router can send a BGP advertisement that does not exactly match items in its own routing table). What are the advantages of each approach?

16.11 Consider a function used to convert between delay and hop-count metrics. Can you find properties of such functions that are sufficient to prevent routing loops. Are your properties necessary as well? (Hint: look at Mills and Braun [1987].)

16.12 Are there circumstances under which an SPF protocol can form routing loops? Hint: think of best-effort delivery.

16.13 Build an application program that sends a request to a router running RIP and displays the routes returned.

16.14 Read the RIP specification carefully. Can routes reported in a response to a query differ from the routes reported by a routing update message? If so how?

16.15 Read the OSPF specification carefully. How can a manager use the virtual link facility?

16.16 OSPF allows managers to assign many of their own identifiers, possibly leading to duplication of values at multiple sites. Which identifier(s) may need to change if two sites running OSPF decide to merge?

16.17 Compare the version of OSPF available under 4BSD UNIX to the version of RIP for the same system. What are the differences in source code size? Object code size? Data storage size? What can you conclude?

16.18 Can you use ICMP redirect messages to pass routing information among *interior* routers? Why or why not?

16.19 Write a program that takes as input a description of your organization's internet, uses RIP queries to obtain routes from the routers, and reports any inconsistencies.

16.20 If your organization runs *gated*, obtain a copy of the configuration files and explain the meaning of each item.

17

Internet Multicasting

17.1 Introduction

Earlier chapters define the original IP classful addressing scheme and extensions such as subnetting and classless addressing. This chapter explores an additional feature of the IP addressing scheme that permits efficient multipoint delivery of datagrams. We begin with a brief review of the underlying hardware support. Later sections describe IP addressing for multipoint delivery and protocols that routers use to propagate the necessary routing information.

17.2 Hardware Broadcast

Many hardware technologies contain mechanisms to send packets to multiple destinations simultaneously (or nearly simultaneously). Chapter 2 reviews several technologies and discusses the most common form of multipoint delivery: *broadcasting*. Broadcast delivery means that the network delivers one copy of a packet to each destination. On bus technologies like Ethernet, broadcast delivery can be accomplished with a single packet transmission. On networks composed of switches with point-to-point connections, software must implement broadcasting by forwarding copies of the packet across individual connections until all switches have received a copy.

With most hardware technologies, a computer specifies broadcast delivery by sending a packet to a special, reserved destination address called the *broadcast address*. For example, Ethernet hardware addresses consist of 48-bit identifiers, with the all 1s address used to denote broadcast. Hardware on each machine recognizes the machine's hardware address as well as the broadcast address, and accepts incoming packets that have either address as their destination.

The chief disadvantage of broadcasting arises from its demand on resources — in addition to using network bandwidth, each broadcast consumes computational resources on all machines. For example, it would be possible to design an alternative internet protocol suite that used broadcast to deliver datagrams on a local network and relied on IP software to discard datagrams not intended for the local machine. However, such a scheme would be extremely inefficient because all computers on the network would receive and process every datagram, even though a machine would discard most of the datagrams that arrived. Thus, the designers of TCP/IP used unicast routing and address binding mechanisms like ARP to eliminate broadcast delivery.

17.3 Hardware Origins Of Multicast

Some hardware technologies support a second, less common form of multi-point delivery called *multicasting*. Unlike broadcasting, multicasting allows each system to choose whether it wants to participate in a given multicast. Typically, a hardware technology reserves a large set of addresses for use with multicast. When a group of machines want to communicate, they choose one particular *multicast address* to use for communication. After configuring their network interface hardware to recognize the selected multicast address, all machines in the group will receive a copy of any packet sent to that multicast address.

At a conceptual level, multicast addressing can be viewed as a generalization of all other address forms. For example, we can think of a conventional *unicast address* as a form of multicast addressing in which there is exactly one computer in the multicast group. Similarly, we can think of directed broadcast addressing as a form of multicasting in which all computers on a particular network are members of the multicast group. Other multicast addresses can correspond to arbitrary sets of machines.

Despite its apparent generality, multicasting cannot replace conventional forms because there is a fundamental difference in the underlying mechanisms that implement forwarding and delivery. Unicast and broadcast addresses identify a computer or a set of computers attached to one physical segment, so forwarding depends on the network topology. A multicast address identifies an arbitrary set of listeners, so the forwarding mechanism must propagate the packet to all segments. For example, consider two LAN segments connected by an adaptive bridge that has learned host addresses. If a host on segment *1* sends a unicast frame to another host on segment *1*, the bridge will not forward the frame to segment *2*. If a host uses a multicast address, however, the bridge will forward the frame. Thus, we can conclude:

> *Although it may help us to think of multicast addressing as a generalization that subsumes unicast and broadcast addresses, the underlying forwarding and delivery mechanisms can make multicast less efficient.*

17.4 Ethernet Multicast

Ethernet provides a good example of hardware multicasting. One-half of the Ethernet addresses are reserved for multicast — the low-order bit of the high-order octet distinguishes conventional unicast addresses (*0*) from multicast addresses (*1*). In dotted hexadecimal notation†, the multicast bit is given by:

$$01.00.00.00.00.00_{16}$$

When an Ethernet interface board is initialized, it begins accepting packets destined for either the computer's hardware address or the Ethernet broadcast address. However, device driver software can reconfigure the device to allow it to also recognize one or more multicast addresses. For example, suppose the driver configures the Ethernet multicast address:

$$01.5E.00.00.00.01_{16}$$

After the configuration, an interface will accept any packet sent to the computer's unicast address, the broadcast address, or that one multicast address (the hardware will continue to ignore packets sent to other multicast addresses). The next sections explain both how IP uses basic multicast hardware and the special meaning of the multicast address

17.5 IP Multicast

IP multicasting is the internet abstraction of hardware multicasting. It follows the paradigm of allowing transmission to a subset of host computers, but generalizes the concept to allow the subset to spread across arbitrary physical networks throughout the internet. In IP terminology, a given subset is known as a *multicast group*. IP multicasting has the following general characteristics:

- *Group address.* Each multicast group is a unique class *D* address. A few IP multicast addresses are permanently assigned by the Internet authority, and correspond to groups that always exist even if they have no current members. Other addresses are temporary, and are available for private use.
- *Number of groups.* IP provides addresses for up to 2^{28} simultaneous multicast groups. Thus, the number of groups is limited by practical constraints on routing table size rather than addressing.
- *Dynamic group membership.* A host can join or leave an IP multicast group at any time. Furthermore, a host may be a member of an arbitrary number of multicast groups.

†Dotted hexadecimal notation represents each octet as two hexadecimal digits with octets separated by periods; the subscript *16* can be omitted only when the context is unambiguous.

- *Use of hardware.* If the underlying network hardware supports multicast, IP uses hardware multicast to send IP multicast. If the hardware does not support multicast, IP uses broadcast or unicast to deliver IP multicast.
- *Inter-network forwarding.* Because members of an IP multicast group can attach to multiple physical networks, special *multicast routers* are required to forward IP multicast; the capability is usually added to conventional routers.
- *Delivery semantics.* IP multicast uses the same best-effort delivery semantics as other IP datagram delivery, meaning that multicast datagrams can be lost, delayed, duplicated, or delivered out of order.
- *Membership and transmission.* An arbitrary host may send datagrams to any multicast group; group membership is only used to determine whether the host receives datagrams sent to the group.

17.6 The Conceptual Pieces

Three conceptual pieces are required for a general purpose internet multicasting system:

1. A multicast addressing scheme
2. An effective notification and delivery mechanism
3. An efficient internetwork forwarding facility

Many goals, details, and constraints present challenges for an overall design. For example, in addition to providing sufficient addresses for many groups, the multicast *addressing scheme* must accommodate two conflicting goals: allow local autonomy in assigning addresses, while defining addresses that have meaning globally. Similarly, hosts need a *notification mechanism* to inform routers about multicast groups in which they are participating, and routers need a *delivery mechanism* to transfer multicast packets to hosts. Again there are two possibilities: we desire a system that makes effective use of hardware multicast when it is available, but also allows IP multicast delivery over networks that do not have hardware support for multicast. Finally a multicast *forwarding facility* presents the biggest design challenge of the three: our goal is a scheme that is both efficient and dynamic — it should route multicast packets along the shortest paths, should not send a copy of a datagram along a path if the path does not lead to a member of the group, and should allow hosts to join and leave groups at any time.

IP multicasting includes all three aspects. It defines IP multicast addressing, specifies how hosts send and receive multicast datagrams, and describes the protocol routers use to determine multicast group membership on a network. The remainder of the chapter considers each aspect in more detail, beginning with addressing.

17.7 IP Multicast Addresses

We said that IP multicast addresses are divided into two types: those that are permanently assigned, and those that are available for temporary use. Permanent addresses are called *well-known*; they are used for major services on the global Internet as well as for infrastructure maintenance (e.g., multicast routing protocols). Other multicast addresses correspond to *transient multicast groups* that are created when needed and discarded when the count of group members reaches zero.

Like hardware multicasting, IP multicasting uses the datagram's destination address to specify that a particular datagram must be delivered via multicast. IP reserves class *D* addresses for multicast; they have the form shown in Figure 17.1.

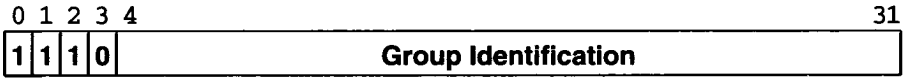


Figure 17.1 The format of class D IP addresses used for multicasting. Bits *4* through *31* identify a particular multicast group.

The first *4* bits contain *1110* and identify the address as a multicast. The remaining *28* bits specify a particular multicast group. There is no further structure in the group bits. In particular, the group field is not partitioned into bits that identify the origin or owner of the group, nor does it contain administrative information such as whether all members of the group are on one physical network.

When expressed in dotted decimal notation, multicast addresses range from

224.0.0.0 through 239.255.255.255

However, many parts of the address space have been assigned special meaning. For example, the lowest address, 224.0.0.0, is reserved; it cannot be assigned to any group. Furthermore, the remaining addresses up through 224.0.0.255 are devoted to multicast routing and group maintenance protocols; a router is prohibited from forwarding a datagram sent to any address in that range. Figure 17.2 shows a few examples of permanently assigned addresses.

Address	Meaning
224.0.0.0	Base Address (Reserved)
224.0.0.1	All Systems on this Subnet
224.0.0.2	All Routers on this Subnet
224.0.0.3	Unassigned
224.0.0.4	DVMRP Routers
224.0.0.5	OSPFIGP All Routers
224.0.0.6	OSPFIGP Designated Routers
224.0.0.7	ST Routers
224.0.0.8	ST Hosts
224.0.0.9	RIP2 Routers
224.0.0.10	IGRP Routers
224.0.0.11	Mobile-Agents
224.0.0.12	DHCP Server / Relay Agent
224.0.0.13	All PIM Routers
224.0.0.14	RSVP-Encapsulation
224.0.0.15	All-CBT-Routers
224.0.0.16	Designated-Sbm
224.0.0.17	All-Sbms
224.0.0.18	VRRP
224.0.0.19 through 224.0.0.255	Unassigned
224.0.1.21	DVMRP on MOSPF
224.0.1.84	Jini Announcement
224.0.1.85	Jini Request
239.192.0.0 through 239.251.255.255	Scope restricted to one organization
239.252.0.0 through 239.255.255.255	Scope restricted to one site

Figure 17.2 Examples of a few permanent IP multicast address assignments. Many other addresses have specific meanings.

We will see that two of the addresses in the figure are especially important to the multicast delivery mechanism. Address 224.0.0.1 is permanently assigned to the *all systems group*, and address 224.0.0.2 is permanently assigned to the *all routers group*. The *all systems* group includes all hosts and routers on a network that are participating in IP multicast, whereas the *all routers* group includes only the routers that are participating. In general, both of these groups are used for control protocols and not for the

normal delivery of data. Furthermore, datagrams sent to these addresses only reach machines on the same local network as the sender; there are no IP multicast addresses that refer to all systems in the internet or all routers in the internet.

17.8 Multicast Address Semantics

IP treats multicast addresses differently than unicast addresses. For example, a multicast address can only be used as a destination address. Thus, a multicast address can never appear in the source address field of a datagram, nor can it appear in a source route or record route option. Furthermore, no ICMP error messages can be generated about multicast datagrams (e.g., destination unreachable, source quench, echo reply, or time exceeded). Thus, a ping sent to a multicast address will go unanswered.

The rule prohibiting ICMP errors is somewhat surprising because IP routers do honor the time-to-live field in the header of a multicast datagram. As usual, each router decrements the count, and discards the datagram (without sending an ICMP message) if the count reaches zero. We will see that some protocols use the time-to-live count as a way to limit datagram propagation.

17.9 Mapping IP Multicast To Ethernet Multicast

Although the IP multicast standard does not cover all types of network hardware, it does specify how to map an IP multicast address to an Ethernet multicast address. The mapping is efficient and easy to understand:

> *To map an IP multicast address to the corresponding Ethernet multicast address, place the low-order 23 bits of the IP multicast address into the low-order 23 bits of the special Ethernet multicast address* $01.00.5E.00.00.00_{16}$.

For example, IP multicast address 224.0.0.2 becomes Ethernet multicast address $01.00.5E.00.00.02_{16}$.

Interestingly, the mapping is not unique. Because IP multicast addresses have 28 significant bits that identify the multicast group, more than one multicast group may map onto the same Ethernet multicast address at the same time. The designers chose this scheme as a compromise. On one hand, using 23 of the 28 bits for a hardware address means most of the multicast address is included. The set of addresses is large enough so the chances of two groups choosing addresses with all low-order 23 bits identical is small. On the other hand, arranging for IP to use a fixed part of the Ethernet multicast address space makes debugging much easier and eliminates interference between IP and other protocols that share an Ethernet. The consequence of this design is that some multicast datagrams may be received at a host that are not destined for that host. Thus, the IP software must carefully check addresses on all incoming datagrams and discard any unwanted multicast datagrams.

17.10 Hosts And Multicast Delivery

We said that IP multicasting can be used on a single physical network or throughout an internet. In the former case, a host can send directly to a destination host merely by placing the datagram in a frame and using a hardware multicast address to which the receiver is listening. In the latter case, special *multicast routers* forward multicast datagrams among networks, so a host must send the datagram to a multicast router. Surprisingly, a host does not need to install a route to a multicast router, nor does the host's default route need to specify one. Instead, the technique a host uses to forward a multicast datagram to a router is unlike the routing lookup used for unicast and broadcast datagrams — the host merely uses the local network hardware's multicast capability to transmit the datagram. Multicast routers listen for all IP multicast transmissions; if a multicast router is present on the network, it will receive the datagram and forward it on to another network if necessary. Thus, the primary difference between local and nonlocal multicast lies in multicast routers, not in hosts.

17.11 Multicast Scope

The *scope* of a multicast group refers to the range of group members. If all members are on the same physical network, we say that the group's scope is restricted to one network. Similarly, if all members of a group lie within a single organization, we say that the group has a scope limited to one organization.

In addition to the group's scope, each multicast datagram has a scope which is defined to be the set of networks over which a given multicast datagram will be propagated. Informally, a datagram's scope is referred to as its *range*.

IP uses two techniques to control multicast scope. The first technique relies on the datagram's *time-to-live* (*TTL*) field to control its range. By setting the TTL to a small value, a host can limit the distance the datagram will be routed. For example, the standard specifies that control messages, which are used for communication between a host and a router on the same network, must have a TTL of 1. As a consequence, a router never forwards any datagram carrying control information because the TTL expires causing the router to discard the datagram. Similarly, if two applications running on a single host want to use IP multicast for interprocessor communication (e.g., for testing software), they can choose a TTL value of 0 to prevent the datagram from leaving the host. It is possible to use successively larger values of the TTL field to further extend the notion of scope. For example, some router vendors suggest configuring routers at a site to restrict multicast datagrams from leaving the site unless the datagram has a TTL greater than 15. We conclude that it is possible to use the TTL field in a datagram header to provide coarse-grain control over the datagram's scope.

Known as *administrative scoping*, the second technique used to control scoping consists of reserving parts of the address space for groups that are local to a given site or local to a given organization. According to the standard, routers in the Internet are forbidden from forwarding any datagram that has an address chosen from the restricted

space. Thus, to prevent multicast communication among group members from accidentally reaching outsiders, an organization can assign the group an address that has local scope. Figure 17.2 shows examples of address ranges that correspond to administrative scoping.

17.12 Extending Host Software To Handle Multicasting

A host participates in IP multicast at one of three levels as Figure 17.3 shows:

Level	Meaning
0	Host can neither send nor receive IP multicast
1	Host can send but not receive IP multicast
2	Host can both send and receive IP multicast

Figure 17.3 The three levels of participation in IP multicast.

Modifications that allow a host to send IP multicast are not difficult. The IP software must allow an application program to specify a multicast address as a destination IP address, and the network interface software must be able to map an IP multicast address into the corresponding hardware multicast address (or use broadcast if the hardware does not support multicasting).

Extending host software to receive IP multicast datagrams is more complex. IP software on the host must have an API that allows an application program to declare that it wants to join or leave a particular multicast group. If multiple application programs join the same group, the IP software must remember to pass each of them a copy of datagrams that arrive destined for that group. If all application programs leave a group, the host must remember that it no longer participates in the group. Furthermore, as we will see in the next section, the host must run a protocol that informs the local multicast routers of its group membership status. Much of the complexity comes from a basic idea:

Hosts join specific IP multicast groups on specific networks.

That is, a host with multiple network connections may join a particular multicast group on one network and not on another. To understand the reason for keeping group membership associated with networks, remember that it is possible to use IP multicasting among local sets of machines. The host may want to use a multicast application to interact with machines on one physical net, but not with machines on another.

Because group membership is associated with particular networks, the software must keep separate lists of multicast addresses for each network to which the machine attaches. Furthermore, an application program must specify a particular network when it asks to join or leave a multicast group.

17.13 Internet Group Management Protocol

To participate in IP multicast on a local network, a host must have software that allows it to send and receive multicast datagrams. To participate in a multicast that spans multiple networks, the host must inform local multicast routers. The local routers contact other multicast routers, passing on the membership information and establishing routes. We will see later that the concept is similar to conventional route propagation among internet routers.

Before a multicast router can propagate multicast membership information, it must determine that one or more hosts on the local network have decided to join a multicast group. To do so, multicast routers and hosts that implement multicast must use the *Internet Group Management Protocol* (*IGMP*) to communicate group membership information. Because the current version is *2*, the protocol described here is officially known as *IGMPv2*.

IGMP is analogous to ICMP†. Like ICMP, it uses IP datagrams to carry messages. Also like ICMP, it provides a service used by IP. Therefore,

> *Although IGMP uses IP datagrams to carry messages, we think of it as an integral part of IP, not a separate protocol.*

Furthermore, IGMP is a standard for TCP/IP; it is required on all machines that receive IP multicast (i.e., all hosts and routers that participate at level *2*).

Conceptually, IGMP has two phases. Phase 1: When a host joins a new multicast group, it sends an IGMP message to the group's multicast address declaring its membership. Local multicast routers receive the message, and establish necessary routing by propagating the group membership information to other multicast routers throughout the internet. Phase 2: Because membership is dynamic, local multicast routers periodically poll hosts on the local network to determine whether any hosts still remain members of each group. If any host responds for a given group, the router keeps the group active. If no host reports membership in a group after several polls, the multicast router assumes that none of the hosts on the network remain in the group, and stops advertising group membership to other multicast routers.

17.14 IGMP Implementation

IGMP is carefully designed to avoid adding overhead that can congest networks. In particular, because a given network can include multiple multicast routers as well as hosts that all participate in multicasting, IGMP must avoid having all participants generate control traffic. There are several ways IGMP minimizes its effect on the network:

> First, all communication between hosts and multicast routers uses IP multicast. That is, when IGMP messages are encapsulated in an IP datagram for transmission, the IP destination address is a multicast address — routers

†Chapter 9 discusses ICMP, the Internet Control Message Protocol.